

Towards Looped Models Done Right

Part II: Rethinking at Fixed Points

Benhao Huang^{†*}, Chufan Shi^{††}, Junlin Chen[†], Shicheng Wen^{††}

Zhengzhong Liu[†], Eric Xing[†], Xuezhe Ma^{††}

[†]Institute of Foundation Models, [†]USC, ^{*}CMU

 Code: <https://github.com/ifm-ai/xllm-loop>

Date: September 29, 2026

Overview

Every recurrence of a looped language model adds cost in training, decoding, prefill, and reinforcement learning (RL). The closer recurrent states get to fixed points, the less the path to them matters. This enables truncated backpropagation in training; terminal key-value (KV) sharing for decoding with almost no loss in accuracy; a distilled student that prefills up to $1.79\times$ faster; and RL updates that compute gradients from saved rollout states, $2\times$ faster than backpropagating through the replayed trajectory. We therefore improve the two components of training that shape these fixed points: the depth prior and input injection. Fixed-depth training breaks KV sharing, and Huginn’s broad depth prior supports sharing but dilutes supervision at the target depth more than sharing requires; we learn the prior from prediction feedback, with an entropy term that keeps it broad. Existing injection schemes let the state’s component along the input amplify or cancel the injection; we remove this component with orthogonal injection. From 100M to 1.6B parameters, the learned prior and orthogonal injection lower perplexity at every scale relative to Huginn’s prior and existing injection schemes, respectively. At 1.6B, the learned prior with a $3\times$ smaller KV cache matches the downstream average of fixed-depth training with the full cache.

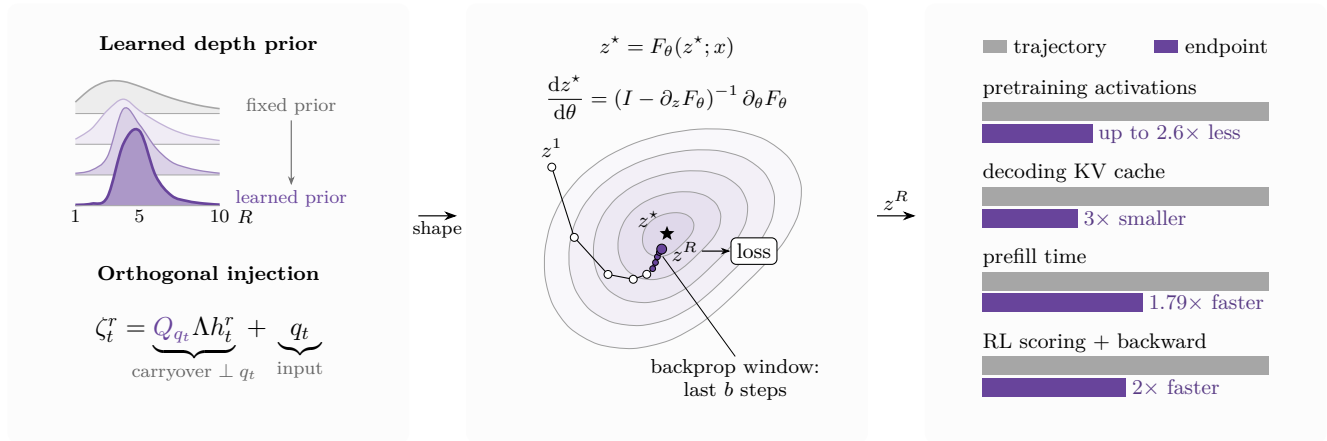


Figure 1: **What shapes fixed points, and what fixed points enable.** Near a fixed point, the recurrence trajectory can be skipped: training backpropagates through only the last b steps, decoding keeps terminal KV banks, a student distills prefill, and RL reuses saved states to score and update (Section 3). The depth prior and input injection shape the fixed points; we learn the prior from prediction feedback and project the carryover off the input (Section 4).

1 Introduction

Looped language models repeatedly apply a shared block of layers [1, 2], but whether they should learn fixed points remains contested. One camp forgoes fixed points and keeps the loop count small and fixed [3–5]. The other camp shapes fixed points [6–8] and loops much deeper. In typical implementations, every loop adds cost: activations to store during training, keys and values (KV) to cache during decoding, compute to prefill each prompt, and a replay to score each rollout during reinforcement learning (RL). In this work, we show that fixed points hold the potential to cut each of these costs (Figure 1), which makes deep looped models cheaper at every stage: pretraining, serving, and post-training.

The fixed-point equation $z^* = F_\theta(z^*; x)$ involves only the endpoint, not the path to it, so once a model’s states are near a fixed point, the endpoint can stand in for the full trajectory (Section 3). Reusing terminal KV caches across loops tests this claim: it barely changes the zero-shot GSM8K accuracy of Huginn [2] but collapses that of Ouro [5] at four loops [9]. The split tracks convergence: Huginn’s states approach fixed points [10]; Ouro’s do not.

Similarly, the same fixed point gives each remaining cost a shortcut. In pretraining, backpropagating through only the last few loops approximates the equilibrium gradient [11, 12] without the warm-up that implicit gradients require. In decoding, *terminal KV sharing* keeps only each prefix token’s terminal KV (Figure 3b) at little cost in performance. In prefilling, a student distilled to predict the fixed point directly replaces the loops (Figure 3c). In RL, scoring rollouts from saved fixed-point states avoids replaying the loops. Each shortcut removes a depth-dependent cost, so its savings grow with every added loop. Looped models already take the first two [2, 5, 13]; fixed points explain why they work and yield the other two.

Realizing these benefits requires training that shapes the fixed points. Two components do the shaping, and current choices for each fall short (Section 4). The first is the *depth prior*, the distribution of training depths. Because tokens converge at different depths (Figure 2), the sampled depth decides whether training sees the converged prefix context that terminal KV sharing supplies at inference (Figure 3a). A model trained at a single depth predicts best there but breaks KV sharing; Huginn’s fixed broad prior [2] supports sharing but dilutes supervision at the target depth more than sharing requires. We learn the prior from prediction feedback, which pulls supervision toward depths that predict well, and add an entropy term that keeps it broad enough for sharing. The second component is *input injection* [2, 14–16], which adds the input to the state at every recurrence. Huginn and Parcae leave the state’s component along the input unconstrained; depending on its sign, this component amplifies or cancels the injection. Our orthogonal input injection (OrthoInj) removes this component, so every recurrence receives the input at the same strength.

- Fixed points explain the two existing shortcuts (Section 3): truncated backpropagation alone shapes the fixed points that terminal KV sharing relies on. Fixed-depth training with full backpropagation through time does not; at 1.6B, sharing drops its GSM8K accuracy from 50.6 to 21.2. With tolerant fixed points, sharing cuts the KV cache $3\times$ at five recurrences, keeping the entropy-regularized learned-prior models within 0.4 points of their full-cache downstream average.
- Fixed points yield two new shortcuts. Distilled prefill (Section 5.5), trained on a quarter of the teacher’s pretraining tokens, is up to $1.79\times$ faster on 8K prompts and trails the teacher’s downstream average by 1.0–4.5 points; at matched latency, it scores 0.5–0.9 points above the teacher prefilling with two recurrences. Rollout-state reuse (Section 3.1) halves the scoring and backward time of each RL update relative to replaying the loops with full backpropagation through time, and stays within 1.6 points of that baseline’s GSM8K pass@1.

- The learned depth prior keeps prediction quality under terminal KV sharing (Section 5.2): from 100M to 1.6B, it lowers perplexity and raises the downstream average over Huginn’s fixed prior, and at 1.6B, with a $3\times$ smaller KV cache, it even matches the full-cache downstream average of fixed-depth training.
- OrthoInj improves on the strongest injection baseline, Parcae’s (Section 5.3). From 100M to 1.6B, it trains to a lower loss, lowers validation perplexity by 0.3–1.4%, and raises the downstream average.

2 Fixed Points in Looped Language Models

Recurrent states in looped language models tend to approach fixed points as recurrence depth grows [2, 10].¹ For an input $x = (x_1, \dots, x_N)$, the state after r recurrent updates is $z^r = (\mathbf{H}^r, C^r)$: hidden states $\mathbf{H}^r = (h_1^r, \dots, h_N^r)$ and the KV banks C^r written on update r , one per physical attention layer. The shared update map gives $z^{r+1} = F_\theta(z^r; x)$, and z^* is a *fixed point* if $F_\theta(z^*; x) = z^*$. We use Huginn [2] for experiments. A prelude of non-shared blocks embeds the input as $e = P_\theta(x)$. A recurrent core of shared blocks starts from a random state z^0 and iterates $z^r = F_\theta(z^{r-1}; x)$ for $r = 1, \dots, R$, injecting e at every recurrence (Section 4.2); we write $z_\theta^R(x)$ for the final state. A coda of non-shared blocks maps $z_\theta^R(x)$ to next-token logits.

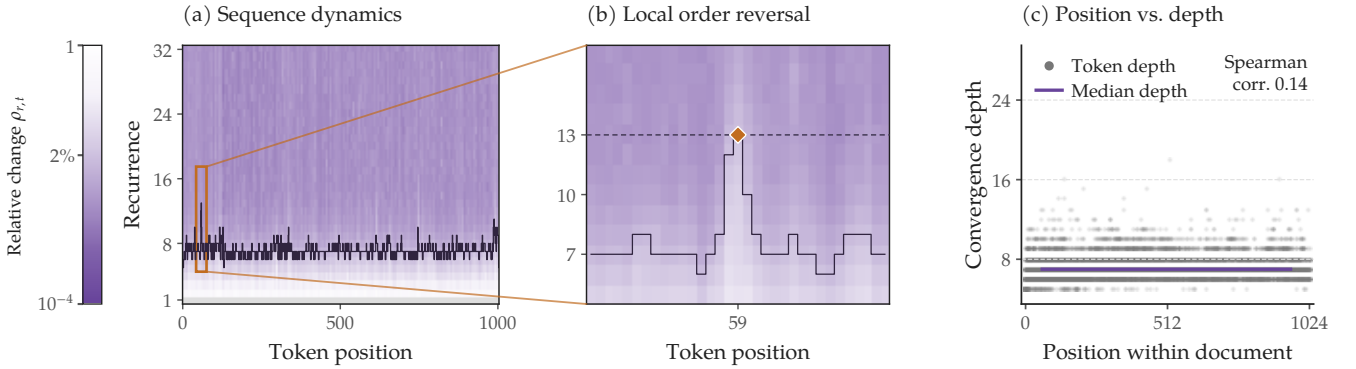


Figure 2: **Heterogeneous token convergence in Huginn.** (a) Relative hidden-state changes $\rho_{r,t}$, with convergence threshold $\tau = 2\%$. (b) Later tokens can approach approximate fixed points before earlier ones. (c) Position versus convergence depth; the purple line shows medians over 128-position intervals. Details in Appendix C.4.

We measure convergence by the tokenwise relative change $\rho_{r,t} = \|h_t^r - h_t^{r-1}\|_2 / (\|h_t^{r-1}\|_2 + \epsilon)$, with $\epsilon = 10^{-8}$. Token t is *converged from depth* r if $\rho_{r',t} < \tau$ for all $r \leq r' \leq R_{\text{obs}}$, the observation horizon; its *convergence depth* is the smallest such r .

Convergence in Huginn is *heterogeneous*: tokens in a sequence converge at different depths, not necessarily in causal order (Figure 2b), and convergence depth correlates only weakly with position (Spearman correlation 0.14; Figure 2c). A prefix $x_{<t}$ is a *converged prefix* from depth r if every token in it is converged from r ; Section 3.2 relates such prefixes to KV sharing.

3 What Fixed-Point Shaping Enables

Fixed-point shaping trains a looped model with supervision near the fixed points of its recurrence [8, 14]. At a fixed point, the endpoint can stand in for the trajectory. Conventional looped language models pay for every recurrence in both training and inference; fixed-point shaping decouples both costs from depth.

¹“Fixed points” here include approximately stationary states at finite depths.

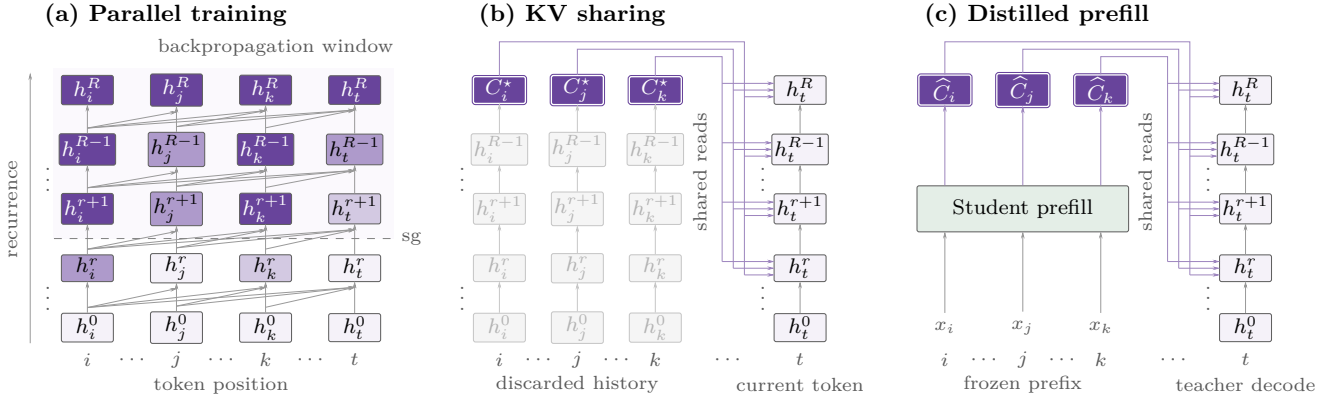


Figure 3: **The endpoint stands in for the recurrent trajectory.** Recurrence runs upward; ellipses mark omitted positions and depths. (a) In training, each update reads prefix states from the previous depth; darker states are closer to convergence (as in Figure 2). Prefix tokens converge at different depths (Section 2), so the sampled depth R decides whether reads inside the backpropagation window see converged context; gradients stop at sg. (b) Decoding reads only the terminal prefix KV banks $C_{<t}^*$ and discards the others, with nearly unchanged downstream accuracy. (c) During prefill, a distilled student replaces the recurrence and supplies approximate terminal banks $\hat{C}_{<t}$.

3.1 Training: Endpoint Gradients

Full backpropagation through time (BPTT) differentiates through all R recurrences, so its cost grows with depth. When near a fixed point, the gradient can instead be computed from the endpoint.

Pretraining: truncated backpropagation. At a fixed point z^* , let $J_\star = \partial_z F_\theta(z^*; x)$ and $B_\star = \partial_\theta F_\theta(z^*; x)$. If F_θ is continuously differentiable and the spectral radius of $J_\star$ is below one, implicit differentiation gives the *equilibrium sensitivity* [14] $D_\star \equiv dz^*/d\theta$, and truncating its Neumann series gives D_b [11, 12] (derivation in Appendix A.2):

$$D_\star = (I - J_\star)^{-1} B_\star = \sum_{k=0}^{\infty} J_\star^k B_\star, \quad D_b = \sum_{k=0}^{b-1} J_\star^k B_\star, \quad \tilde{D}_b = \sum_{k=0}^{b-1} \left(\prod_{j=1}^k J_{R-j} \right) B_{R-1-k}. \quad (1)$$

$D_\star$ depends only on z^* , and it equals the $R \rightarrow \infty$ limit of full BPTT when that trajectory converges. Its truncated Neumann estimate D_b has a bias $D_\star - D_b = J_\star^b D_\star$ that decays geometrically in b . Both, however, are valid only once a stable fixed point exists. Applied from initialization, such endpoint gradients destabilize training (Appendix C.3), so implicit models first warm up with unrolled backpropagation [14, 17, 18].

Truncated BPTT (TBPTT) instead differentiates through only the last b recurrences [19, 20], so its memory and compute scale with b rather than R . Its gradient $\tilde{D}_b$ evaluates the Jacobians $J_r = \partial_z F_\theta(z^r; x)$ and $B_r = \partial_\theta F_\theta(z^r; x)$ at visited states z^r rather than at z^* . This frees TBPTT from needing a fixed point: from any state, $\tilde{D}_b$ is the exact gradient of the loss through the last b recurrences with the earlier state held fixed, so every update descends a well-defined objective. As the visited states converge to z^* , $\tilde{D}_b$ approaches D_b regardless of R . TBPTT is therefore as stable as full BPTT early in training and recovers the fixed-point gradient once fixed points form, using a single estimator with no warm-up phase or switch.

Furthermore, truncation alone shapes fixed points. A fixed point has two testable properties: (i) extra recurrences leave the state unchanged, and (ii) terminal KV sharing costs little (Lemma 1). At fixed depth $R = 5$, full BPTT fails both: extrapolating to $R = 64$ raises perplexity by over 25%, and terminal KV sharing raises it by 21%. In contrast, TBPTT through the last two recurrences ($b = 2$) raises perplexity by at most

Algorithm 1: Policy update from saved rollout states (Neumann- $(b - 1)$). A single forward pass through the recurrent block records its graph $\mathcal{G}$ (line 2); the Neumann loop (lines 6–9) and the parameter gradient (line 10) reuse $\mathcal{G}$ without running the block again. In code, lines 6–9 run as a backward hook on $\tilde{z}$.

Require: saved endpoints z^R of the sampled sequences x (no gradient), advantages, window b

```

1:  $\bar{z} \leftarrow \text{stopgrad}(z^R)$ , marked as requiring gradients
2:  $f \leftarrow F_\theta(\bar{z}; x)$ , recording its autograd graph  $\mathcal{G}$  ▷ the only pass through the recurrent block
3:  $\tilde{z} \leftarrow \bar{z} + (f - \text{stopgrad}(f))$  ▷ forward value  $z^R$ 
4:  $\mathcal{L} \leftarrow$  policy-gradient loss of the coda readout of  $\tilde{z}$  on the sampled tokens
5:  $g \leftarrow \partial \mathcal{L} / \partial \tilde{z}$  ▷ backward through the readout
6:  $v \leftarrow g$ 
7: for  $k = 1, \dots, b - 1$  do
8:    $v \leftarrow g + J_R^\top v$  ▷ backward through  $\mathcal{G}$  to  $\bar{z}$ : autograd.grad(f,  $\bar{z}$ , v),  $\mathcal{G}$  reused
9: end for
10: backpropagate  $v$  through  $\mathcal{G}$  to  $\theta$  ▷  $\mathcal{G}$  reused once more:  $\theta$  receives  $B_R^\top v = (\partial_\theta F_\theta)^\top v$ 
11: optimizer step on  $\theta$  with these and the readout gradients; discard  $z^R$ 

```

2% and 3%, respectively (Appendix C.3). With sampled depths enabled, training with $b \geq 4$ also matches full BPTT within 1% perplexity at 400M (Table 17 in Appendix C.3).

Post-training: reusing rollout states for RL. RL usually runs the recurrent forward pass twice: once without gradients to sample rollouts, and again with gradients to score them. Near a fixed point, the second pass need not replay the trajectory. Full BPTT computes $\tilde{D}_R$ of equation (1); when every visited state is close to z^* , its Jacobians are close to J_* and B_* , so $\tilde{D}_R$ approaches D_* , whose truncated Neumann series can be evaluated at the last state z^R :

$$\underbrace{\tilde{D}_R = \sum_{k=0}^{R-1} \left(\prod_{j=1}^k J_{R-j} \right) B_{R-1-k}}_{\text{full BPTT: all } R \text{ recurrences}} \approx \underbrace{\sum_{k=0}^{b-1} J_R^k B_R}_{\text{reuse: endpoint } z^R \text{ only}} \quad (2)$$

The update is on-policy, so the endpoints z^R saved during rollout equal those a fresh forward pass would produce. Reuse applies the recurrent block once at z^R and evaluates the right side of equation (2) by looping the backward pass $b - 1$ times over that one recorded recurrence, $v \leftarrow g + J_R^\top v$ (Neumann- $(b - 1)$, Algorithm 1). Full BPTT instead runs and backpropagates through all R recurrences, and recomputation reruns them without gradients to rebuild z^R . The update cost and memory of reuse are thus independent of R , though rollout generation still takes most of the training time (Section 5.4).

3.2 Inference: Endpoint Caching and Distillation

Decoding: cache only terminal KV banks. The KV cache of conventional looped models grows linearly with recurrence depth. *Terminal KV sharing* removes this growth: each prefix token keeps only its terminal KV, and every recurrence of later tokens attends to it (Figure 3b).² This requires a model that predicts well from terminal context.

However, teacher-forced training alone does not match this setting exactly. It updates all tokens of a sequence in parallel, so recurrence r of the current token reads prefix banks from depth r (Figure 3a); sharing instead supplies terminal banks from the first recurrence. This gap is small when the prefix banks change little across depth and the current token’s update contracts, so an error introduced by early context decays over later recurrences. Sampled training depths push the model toward this regime: deep draws let

²Decoding reads only terminal banks, so prefill and decoding can run at separate recurrence budgets (Appendix C.6); prefill gains saturate at moderate depth while decoding gains persist deeper.

supervised recurrences read nearly settled prefix banks, and supervising many depths penalizes states that drift as recurrence continues (Figure 4).

Lemma 1 formalizes the limiting case, in which the prefix banks converge. For the current token t , let h^r denote its state at recurrence r , $C_{<t}^r$ the prefix-dependent input to its update (including the prefix’s per-layer KV banks), and $f_\theta(h; C)$ the hidden-state component of F_θ , as a function of state h and context C . By causality, $\{C_{<t}^r\}$ does not depend on the current token’s states, so the prefix enters the current token’s iteration as an external input sequence.

Lemma 1 (Joint iteration and frozen-prefix decoding, proof in Appendix A.1). *Fix the parameters and the input sequence, and suppose $C_{<t}^r \rightarrow C_{<t}^*$. Let $\mathcal{H} \subseteq \mathbb{R}^d$ be nonempty and closed, and let $\mathcal{C} = \{C_{<t}^r\}_{r \geq 0} \cup \{C_{<t}^*\}$. Suppose that for every $C \in \mathcal{C}$, $f_\theta(\cdot; C)$ maps $\mathcal{H}$ into itself and is κ -contractive with $\kappa < 1$, and that for every $h \in \mathcal{H}$, $f_\theta(h; \cdot)$ is β -Lipschitz on $\mathcal{C}$. Then, from any initial states in $\mathcal{H}$, the joint and frozen-prefix iterations*

$$h^{r+1} = f_\theta(h^r; C_{<t}^r), \quad \tilde{h}^{r+1} = f_\theta(\tilde{h}^r; C_{<t}^*) \quad (3)$$

both converge to the unique fixed point $h^ = f_\theta(h^*; C_{<t}^*)$ in $\mathcal{H}$. Consequently, any readout continuous in state and context yields the same limiting next-token distribution under both iterations.*

In the limit, therefore, sequence-parallel supervision near fixed points transfers to decoding with terminal KV sharing. Empirically, at the training depth $R = 5$, sharing keeps 4 of 12 banks and raises perplexity over the full cache by at most 0.85% (Appendix C.5).

Prefill: distill the recurrent endpoint. Because decoding reads only terminal KV banks, prefill can be distilled into a student that predicts only the recurrent endpoint; without terminal sharing, the student would have to reproduce the KV banks of every recurrence [21]. The student is a lightweight network S_ψ that replaces the iterated recurrent core during prefill; the prelude and coda are reused from the teacher. From the prelude output $e = P_\theta(x)$, the student predicts $\hat{z} = S_\psi(e) \approx z_\theta^R(x)$; prefill then applies one teacher recurrence and the coda to $\hat{z}$ and retains each physical attention layer’s KV bank (Figure 3c). Because fixed-point shaping makes the endpoint an approximate fixed point, one recurrence from a nearby $\hat{z}$ yields banks close to the teacher’s terminal banks. Decoding is unchanged: the teacher runs at its original depth with terminal KV sharing.

We train the student against depth- R targets computed online from the frozen teacher, using normalized hidden-state regression [22] plus a next-token KL term:

$$\mathcal{L}_{\text{distill}}(\psi) = \mathbb{E}_x \left[\frac{\|\hat{z} - z_\theta^R(x)\|_F^2}{Nd\sigma_T^2} + \frac{1}{N} \sum_{t=1}^N \text{KL}(p_t \| \hat{p}_t) \right], \quad (4)$$

where N is the prompt length, d is the hidden width, and σ_T^2 is a fixed teacher-state energy calibrated on training data, so the first term is a per-coordinate squared error on the teacher’s scale. The second term compares the teacher’s next-token distribution p_t at depth R with the distribution $\hat{p}_t$ obtained by applying the frozen coda and output head to $\hat{z}$. We initialize the student from the teacher’s recurrent core, so before training it approximately outputs the teacher’s state after one recurrence; equation (4) then moves this output toward the depth- R endpoint (Appendix C.7.1).

Section 5.5 evaluates distilled prefill. With a suitable loss, the student may also be able to serve as a standalone language model or a drafter for speculative decoding; we leave these uses, and possible connections between recurrent state refinement and flow-based language modeling, to future work.

Algorithm 2: Depth-prior learning. The prior learns from the detached model CE (line 3) and adds no forward pass. The advantage (line 5) centers the reward with EMA statistics (lines 9–11) to reduce variance and rescales it to CE scale, so λ_H and λ_m need no retuning across reward designs (Appendix C.1).

Require: θ_n, ϕ_n , EMA statistics at $n - 1$; $\nu, \lambda_H, \lambda_m, \bar{R}, b_{\max}$

```

1:  $p_{\phi_n} \leftarrow \text{softmax}(\phi_n)$ 
2: for microbatch  $i$  do
3:    $r_i \sim p_{\phi_n}$ ;  $\ell_i \leftarrow \text{CE}(\theta_n, r_i)$ ;  $u_i \leftarrow \exp(-\ell_i)$   $\triangleright \ell_i$  detached
4:   accumulate CE gradients through the final  $\min(r_i, b_{\max})$  recurrences
5:    $A_i \leftarrow s_{n-1}(u_i - \hat{u}_{n-1})$ 
6: end for
7:  $\mathcal{J}_n \leftarrow -\hat{\mathbb{E}}_n[\text{stopgrad}(A_i) \log p_{\phi_n}(r_i)] - \lambda_H H(p_{\phi_n}) + \lambda_m (\mathbb{E}_{p_{\phi_n}}[R] - \bar{R})^2$ 
8:  $\theta_{n+1} \leftarrow \text{ModelStep}(\theta_n)$ ;  $\phi_{n+1} \leftarrow \text{Adam}(\phi_n, \nabla_{\phi_n} \mathcal{J}_n)$ 
9:  $\text{EMA}_n[\chi] \leftarrow \nu \text{EMA}_{n-1}[\chi] + (1 - \nu) \hat{\mathbb{E}}_n[\chi]$ ,  $\chi \in \{\ell, u, \ell^2, u^2\}$ 
10:  $\sigma_{\chi, n} \leftarrow \max\left\{\sqrt{\text{EMA}_n[\chi^2] - \text{EMA}_n[\chi]^2}, \epsilon\right\}$ ,  $\chi \in \{\ell, u\}$ 
11:  $\hat{u}_n \leftarrow \text{EMA}_n[u]$ ;  $s_n \leftarrow \sigma_{\ell, n} / \sigma_{u, n}$ 

```

4 Improving Fixed-Point Shaping

The benefits in Section 3 depend on how training shapes fixed points. Two common techniques do this shaping: depth sampling sets where supervision lands, and input injection sets what a fixed point retains about the input. We improve both (Section 4.1 and Section 4.2).

4.1 Learning a Depth Prior for Fixed-Point Training

The sampled training depth sets how converged the states are when supervision arrives, and thus how well the model tolerates terminal KV sharing (Section 3.2). Huginn [2] samples the training depth from a shifted Poisson–log-normal (PLN) distribution:

$$\xi \sim \mathcal{N}(\mu, \sigma_\xi^2), \quad K \mid \xi \sim \text{Poisson}(\exp(\xi)), \quad R = K + 1, \quad (5)$$

where $\sigma_\xi = \frac{1}{2}$ and $\mu = \log(\bar{R} - 1) - \sigma_\xi^2/2$, so that $\mathbb{E}[R]$ equals the target mean $\bar{R} > 1$. Under a fixed training budget, **the choice of depth prior trades prediction quality at the target depth against fixed-point shaping** (Figure 4). Fixed-depth training concentrates supervision at one depth and predicts better there. PLN sampling spreads supervision across depths and yields the fixed-point behavior that supports KV sharing (Table 2) and test-time scaling.³

We therefore *learn the depth prior*: it spreads supervision across depths but adapts its allocation to training feedback, balancing prediction quality against fixed-point shaping at equal mean depth.

Learned prior. We replace the PLN prior with a categorical prior $p_\phi = \text{softmax}(\phi)$ over depths $\mathcal{R} = \{1, \dots, R_{\max}\}$, with learnable logits ϕ . At each optimizer step n , every microbatch draws a depth $r \sim p_{\phi_n}$ and receives supervision at that depth. We learn p_ϕ from this feedback by minimizing

$$\mathcal{J}_{\text{prior}}(\phi) = \underbrace{-\hat{\mathbb{E}}_n[\text{stopgrad}(A_n) \log p_\phi(r)]}_{\text{prediction quality}} - \underbrace{\lambda_H H(p_\phi)}_{\text{depth diversity}} + \underbrace{\lambda_m (\mathbb{E}_{p_\phi}[R] - \bar{R})^2}_{\text{budget control}}, \quad (6)$$

where $\hat{\mathbb{E}}_n$ averages over microbatches, weighted by their unmasked positions. The advantage A_n scores the sampled depth. We map the mean next-token CE ℓ to a reward $u = \omega(\text{stopgrad}(\ell))$ with a decreasing function ω ; we use inverse perplexity, $\omega(\ell) = \exp(-\ell)$. The advantage is then $A_n = s_{n-1}(u - \hat{u}_{n-1})$: an

³Appendix C.1 gives the depth distributions and comparison details (Figure 9).

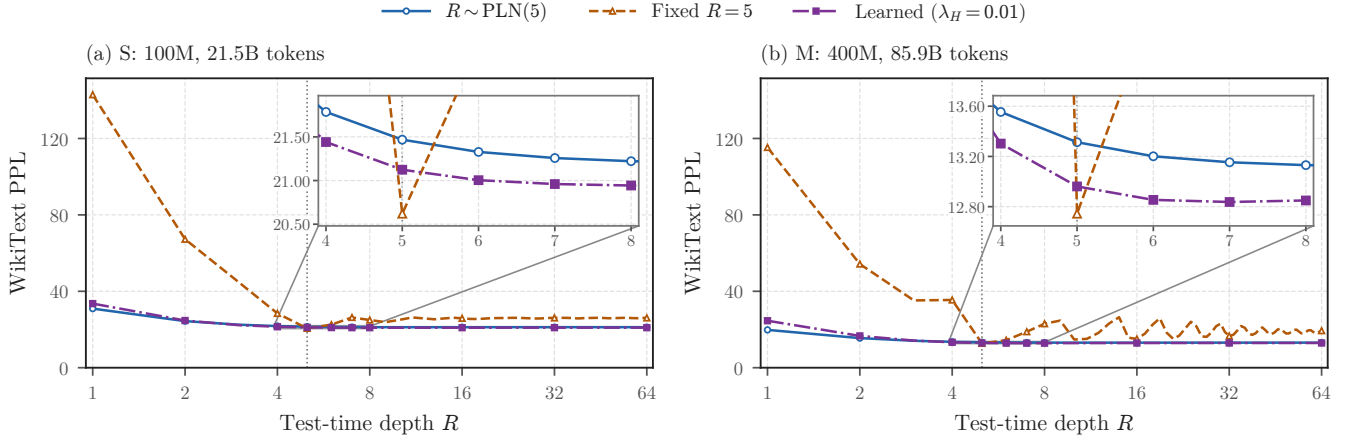


Figure 4: **The learned prior narrows the trade-off between fixed-depth training and PLN sampling.** Native WikiText-103 test PPL across test-time depths at matched training budgets. Fixed-depth training is best at training-matched depth $R = 5$ but degrades sharply at others. PLN sampling performs well across all depths but falls behind at $R = 5$. The learned prior lies between them, narrowing the gap at $R = 5$ while staying stable across depths.

exponential moving average (EMA) baseline $\hat{u}_{n-1}$ centers the reward, and $s_{n-1} = \sigma_{\ell,n-1}/\sigma_{u,n-1}$ rescales it to the scale of CE, following PopArt [23, 24]. A positive A_n thus raises the probability of the sampled depth.

Entropy regularization. Prediction feedback under a budget constraint favors depths that already predict well. Left alone, it concentrates supervision on a narrow range of depths and can weaken fixed-point shaping. The entropy term $H(p_\phi) = -\sum_{r \in \mathcal{R}} p_\phi(r) \log p_\phi(r)$ counters this tendency. Increasing λ_H spreads the depth allocation. Since depths start at 1, the wider allocation extends mainly above $\bar{R}$, toward $R_{\max}$. This trades finite-budget predictive performance for supervision at later depths, where settled prefix context supports KV sharing (Appendix C.1). The coefficient λ_H thus gives a direct knob for balancing performance and memory efficiency.

Budget control. The third term penalizes deviations of the expected depth from the target $\bar{R}$, keeping the allocation near the prescribed forward recurrence budget. Appendix B.3 gives one update step (Algorithm 2) and implementation details.

4.2 Orthogonal Input Injection

Input injection feeds the same embedding q_t into every recurrence [2, 14–16], keeping the fixed points input conditioned. A common additive form is

$$\zeta_t^r = M h_t^r + q_t, \quad \mathbf{H}^{r+1} = \Gamma_\theta(\zeta_1^r, \dots, \zeta_N^r), \quad (7)$$

where h_t^r is the recurrent state at token t and recurrence r , M controls state carryover, and Γ_θ is the recurrent transformer core. Parcae [15] uses a diagonal carryover $M = \Lambda = \text{diag}(\exp(-\Delta \odot \alpha))$ with learned positive vectors Δ and α , so Λ has entries in $(0, 1)$ and $\|\Lambda\|_2 < 1$. It injects the prelude output $e_t = P_\theta(x)_t$ as $q_t = \Delta \odot W e_t$.⁴

Orthogonal injection. In Parcae, the carryover Λh_t^r can have a component along q_t that amplifies or cancels the input, so the input a fixed point retains depends on the state; this can hurt performance (Appendix C.2).

⁴Parcae applies RMSNorm to e_t before injection; OrthoInj drops it. Ablations in Appendix C.2.

OrthoInj removes this component:

$$\zeta_t^r = Q_{q_t} \Lambda h_t^r + q_t, \quad Q_{q_t} = I - \frac{q_t q_t^\top}{\|q_t\|_2^2 + \epsilon}. \quad (8)$$

For $q_t \neq 0$ and $\epsilon = 0$, the component of ζ_t^r along q_t is exactly q_t at every recurrence, including at a fixed point, while the orthogonal part evolves freely; the stabilizer $\epsilon > 0$ makes this approximate. Since $\|Q_{q_t}\|_2 \leq 1$, the projected carryover stays contractive: $\|Q_{q_t} \Lambda\|_2 \leq \|\Lambda\|_2 < 1$.

5 Experiments

We first evaluate the two training changes from Section 4: the learned depth prior and OrthoInj. We then test rollout-state reuse in RL (Section 3.1) and distilled prefill (Section 3.2).

5.1 Setup

Models and training. We build looped models from dense Llama Transformer blocks [25] at Small (S), Medium (M), and Large (L) scales (Table 1). Following Huginn [2], each model has one prelude block, a two-block recurrent core, and one coda block; we write this layout as $1+2 \times R+1$, since R recurrences give a logical depth of $2 + 2R$ blocks. Untied 4 stacks four distinct blocks, matching the physical depth and parameter count. Untied 12 stacks twelve distinct blocks, matching the logical depth of five recurrences, with three times the parameters.

Table 1: **Training scales.**

Scale	Parameters	Tokens (B)
S	100M	21.5
M	400M	85.9
L	1.6B	343.6

We train on a weighted multilingual mixture that includes TxT360 [26], tokenized with the Jais64k tokenizer [27]. Optimization uses AdamW [28] with a warmup–stable–decay schedule [29]: 5% linear warmup, then 10% cosine decay to one-tenth of the peak learning rate. We sweep the peak learning rate at each scale and tune the main configurations separately (Appendix B.1).

Evaluation. We report perplexity (PPL) on MixVal, a held-out set drawn from the TxT360 sources, and on WikiText-103 [30]. Downstream accuracy covers LAMBADA [31], HellaSwag (HS) [32], PIQA [33], ARC-Easy/Challenge [34], OpenBookQA (OBQA) [35], and SciQ [36]. Generation uses GSM8K (GSM) [37], DROP [38], and MBPP+ [39, 40]. Unless noted, we evaluate at $R = 5$ with terminal KV sharing (Section 3.2): prefill caches only the terminal KV of each prompt token, and choice tasks and generation then decode one token at a time. For PPL, we prefill the first half of each 8,192-token sequence and score the second half in one parallel pass, whose tokens read the prefix through terminal banks and one another as in training. Appendix D gives the full protocols.

5.2 Learning the Depth Prior

A good depth prior should give low loss at the target depth and lose little when every iteration shares the terminal iteration’s KV cache. We compare four priors with mean depth 5: fixed-depth training (Fixed $R=5$), Huginn’s fixed Poisson log-normal prior (Fixed PLN-5), and our learned prior with and without the entropy term. Both learned priors start from PLN-5 and cap depth at $R_{\max} = 64$.

Fixed-depth training breaks KV sharing. Fixed-depth training is the worst under forced KV sharing, with the highest perplexity, and sharing also cuts its GSM8K accuracy by over half at M and L (Table 2). It needs full KV to achieve its best performance (Figure 4, Appendix C.1).

The learned prior keeps sharing intact and outperforms Fixed PLN-5. With terminal KV sharing, it lowers validation PPL relative to Fixed PLN-5 by 0.7–1.8%, for at most 1.6% more training FLOPs. It raises

Table 2: **The learned prior far surpasses Untied 4 at the same parameter count and KV cache, and approaches Untied 12 with $3\times$ fewer of both.** The looped models $(1+2\times R+1)$ are evaluated at $R = 5$, a logical depth of 12 blocks, with terminal KV sharing, which caches 4 KV banks, one per physical block, instead of 12. Learned priors start from the Fixed PLN-5 prior. FLOPs are in units of 10^{19} . Generation: GSM8K accuracy, DROP F1, and MBPP+ pass@1. Untied 4 and Untied 12 stack four and twelve distinct blocks and decode with their own full KV caches; Untied 4 matches the looped models’ parameters and cache size, and Untied 12 their logical depth, with $3\times$ their non-embedding parameters and KV cache. Fixed-depth training breaks KV sharing: it loses over half its GSM8K accuracy at M and L relative to the full cache (Appendix Table 8). See Appendix D.3 for evaluation details.

Training		PPL ↓		Downstream (%) ↑								Generation (%) ↑		
Prior	FLOPs	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM	DROP	MBPP+
S scale: (100M, 21.5B tokens)														
Fixed $R=5$	7.384	5.95	20.59	18.24	31.03	62.24	46.13	25.09	15.20	78.20	39.45	1.36	8.33	0.00
Fixed PLN-5	7.292	5.73	20.19	26.41	32.42	61.70	47.56	23.98	16.20	81.00	41.32	1.52	13.29	0.75
Learned, $\lambda_H=0$	7.336	5.67	19.85	26.97	32.23	62.57	45.66	24.83	16.80	81.70	41.54	1.44	12.54	0.88
Learned, $\lambda_H=0.01$	7.328	5.67	19.91	27.36	32.51	63.00	45.50	25.09	17.20	80.90	41.65	1.59	14.08	0.62
Untied 4	3.259	6.57	25.19	23.95	30.26	59.96	44.57	23.21	16.60	77.60	39.45	1.97	11.39	0.12
Untied 12	7.232	4.99	17.24	31.63	34.85	65.13	51.26	26.11	15.80	82.20	43.85	1.90	12.77	2.88
M scale: (400M, 85.9B tokens)														
Fixed $R=5$	92.39	4.21	12.79	34.99	41.83	67.03	56.65	29.61	19.40	88.70	48.32	3.34	14.98	3.25
Fixed PLN-5	91.00	4.00	12.55	39.69	44.64	67.68	56.61	29.78	16.40	88.60	49.06	13.72	17.75	10.38
Learned, $\lambda_H=0$	92.01	3.92	12.24	39.22	45.63	68.88	58.75	28.07	19.40	88.10	49.72	14.33	20.29	13.12
Learned, $\lambda_H=0.01$	92.14	3.93	12.28	38.58	46.17	67.41	57.58	28.75	17.00	88.00	49.07	15.62	17.11	10.38
Untied 4	36.77	4.68	15.73	31.19	37.43	63.71	50.72	27.65	15.80	82.10	44.08	2.35	13.12	2.50
Untied 12	89.95	3.60	11.03	42.19	49.27	70.18	60.19	30.80	19.00	90.50	51.73	16.15	21.11	16.38
L scale: (1.6B, 343.6B tokens)														
Fixed $R=5$	1272	3.24	8.92	48.85	58.19	72.31	66.29	36.01	24.40	92.30	56.90	21.23	27.98	12.75
Fixed PLN-5	1253	3.10	8.76	55.17	60.80	73.56	69.74	38.74	23.80	93.30	59.30	47.61	34.11	27.00
Learned, $\lambda_H=0$	1272	3.12	8.75	53.02	59.79	72.63	68.35	38.82	24.00	92.50	58.44	36.47	31.01	19.38
Learned, $\lambda_H=0.01$	1273	3.07	8.68	55.17	61.57	73.72	71.09	40.44	24.40	93.60	60.00	47.92	34.56	27.25
Untied 4	465	3.66	11.14	40.35	48.63	70.29	62.37	30.63	16.40	88.90	51.08	7.20	18.62	10.50
Untied 12	1233	2.89	7.93	57.62	64.65	74.32	72.22	41.89	25.20	94.40	61.47	48.82	39.82	31.38

the downstream average over Fixed PLN-5 at every scale, is the highest at S and L, and leads all three generation tasks at L. The entropy term is essential at L: without it, GSM8K drops from 47.92 to 36.47. At S scale, training with its final distribution frozen from the first update lowers validation PPL by a further 0.8%, so the learned prior finds a better depth distribution without a search over distributions (Appendix C.1). Appendix C.1 sweeps λ_H (Figure 7), gives training curves and additional ablations.

The learned prior steadily surpasses Untied 4 at the same physical depth, with a lead that grows with scale, and approaches Untied 12 at the same logical depth, with a gap that narrows. With $\lambda_H=0.01$, its downstream-average lead over Untied 4 grows from 2.2 points at S to 5.0 at M and 8.9 at L. Its validation PPL exceeds that of Untied 12 by 13.6% at S, 8.9% at M, and 6.3% at L. At L it trails Untied 12 by 1.5 points on the downstream average and 0.9 on GSM8K, where it leads Untied 4 by 40.7.

5.3 Input Injection

We compare OrthoInj with DEQ-QKV [14], Huginn-Linear [2], and Parcae-Decay [15], all trained with PLN-5 (details in Appendix B.4).

Table 3: **OrthoInj has the highest downstream average and the lowest validation PPL at every scale.** All models train with PLN-5 and are evaluated at $R = 5$ with terminal KV sharing, as in Table 2. Recipes are in Appendix B.4.

Model	PPL ↓		Downstream (%) ↑								Generation (%) ↑		
Injection	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM	DROP	MBPP+
S scale: (100M, 21.5B tokens)													
DEQ-QKV	6.30	23.05	25.17	30.56	60.99	43.31	23.81	15.40	76.10	39.33	2.05	9.42	0.12
Huginn-Linear	5.90	20.78	25.69	31.51	62.35	43.18	24.66	17.20	78.60	40.46	1.90	12.21	0.25
Parcae-Decay	5.75	19.90	26.28	32.14	61.86	46.89	23.81	16.40	81.60	41.28	1.90	12.08	0.25
OrthoInj	5.67	20.02	27.54	32.69	61.97	46.46	24.66	16.40	81.80	41.65	1.90	13.66	1.62
M scale: (400M, 85.9B tokens)													
Huginn-Linear	3.99	12.55	36.56	44.15	68.12	56.69	29.44	17.00	88.70	48.67	16.68	16.71	8.00
Parcae-Decay	3.96	12.45	37.78	44.63	68.12	57.58	27.90	19.40	87.80	49.03	15.01	20.17	8.25
OrthoInj	3.94	12.32	37.82	45.37	68.12	57.07	29.27	18.40	88.10	49.16	13.95	19.38	7.88
L scale: (1.6B, 343.6B tokens)													
Parcae-Decay	3.09	8.72	53.58	60.60	72.80	70.62	39.76	23.40	93.80	59.22	52.92	32.66	30.88
OrthoInj	3.08	8.72	54.51	61.62	72.63	69.53	38.65	24.80	93.10	59.26	51.71	32.77	27.75

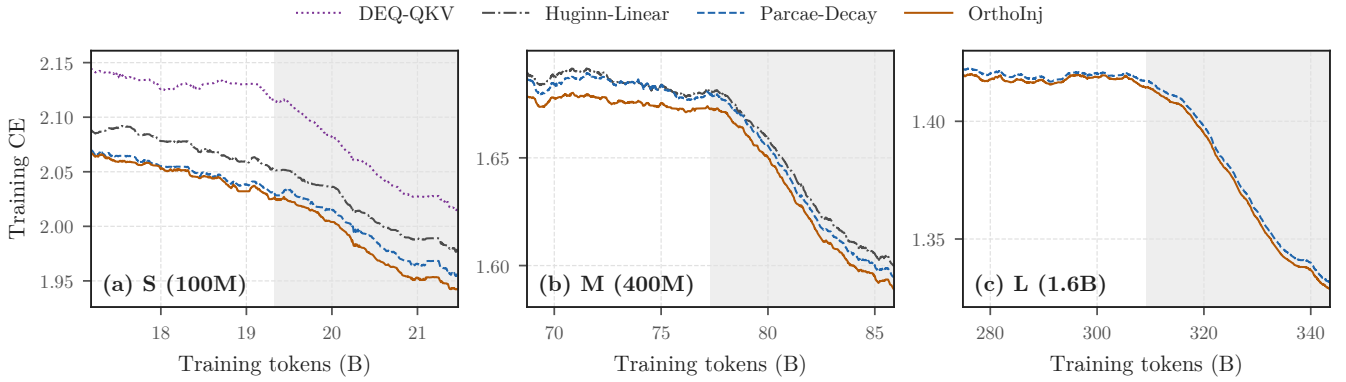


Figure 5: **OrthoInj ends training with the lowest loss at every scale.** Training CE of the Table 3 runs over the final 20% of training, as a centered rolling median over 5% of each run; the gray band marks the learning-rate decay (final 10% of updates). Within each scale, all runs see the same sampled depths.

OrthoInj outperforms existing input injection schemes. With terminal KV sharing, it improves validation PPL over the strongest baseline, Parcae-Decay, and has the highest downstream average at every scale (Table 3). It also ends training with the lowest training loss, validation and WikiText PPL at every scale (Figure 5). Dropping prelude normalization and the projection each help, and their gains add up (Appendix C.2).

5.4 Rollout-State Reuse in RL

We fine-tune the L learned-prior model of Table 2 with Dr. GRPO [41] at $R = 6$, the depth with its best GSM8K accuracy (Appendix C.8). The math run trains on 400 questions from the GSM8K training split and evaluates on 500 GSM8K test questions and on MATH500. The code run trains on 272 MBPP+ tasks and evaluates on 100 held-out MBPP+ tasks. In each run, every update method samples 16 responses per training prompt. We compare reuse, which applies Neumann gradients to saved rollout states (Section 3.1), with BPTT through all recurrences. A control separates state reuse from the Neumann approximation: it computes the same Neumann gradient as reuse but rebuilds the endpoint by replaying all the recurrences

without gradient tracking.

Table 4: **Reusing rollout states cuts scoring and backward time.** The L learned-prior model of Table 2 is fine-tuned with Dr. GRPO at $R = 6$ on GSM8K or MBPP+; the GSM8K runs are also evaluated on MATH500, which RL does not train on. Pass@ k uses eight samples per question. Time: median scoring and backward time per update, excluding rollout and the optimizer step. See Appendix C.8 for training and evaluation details.

Training	RL on GSM8K					RL on MBPP+		
	GSM8K (%) $\uparrow$		MATH500 (%) $\uparrow$		Time (s) $\downarrow$	MBPP+ (%) $\uparrow$		Time (s) $\downarrow$
Update method	Pass@1	Pass@8	Pass@1	Pass@8	Update	Pass@1	Pass@8	Update
L scale: (1.6B, 343.6B tokens)								
Before RL	40.63	77.60	13.40	42.80	—	30.88	63.00	—
Full BPTT	63.20	88.00	18.27	52.00	2.77	40.88	69.00	1.03
Recompute + Neumann-4	60.05	89.60	17.32	50.60	2.09	37.12	69.00	0.65
Reuse + Neumann-4	61.65	90.20	17.82	49.60	1.39	39.38	64.00	0.51
Reuse + Neumann-3	61.30	89.80	18.77	50.20	1.30	38.00	68.00	0.48

Reusing rollout states makes the update $2\times$ faster than full BPTT. Reuse with Neumann-4 cuts scoring and backward time by $1.99\times$ on GSM8K and $2.02\times$ on MBPP+ (Table 4). All four methods raise GSM8K pass@1 from 40.6% to 60–63% and MBPP+ pass@1 from 30.9% to 37–41%. The GSM8K gains transfer to MATH500, which RL does not train on: every method raises pass@1 by 3.9–5.4 points.

Reuse matches full BPTT’s pass@1 within run-to-run variation. Reuse and the recomputation control compute nearly the same gradient (whole-model cosines 0.997–0.999, Appendix C.8), so the gap between their pass@1 measures run-to-run variation: 1.6 points on GSM8K, 0.5 on MATH500, and 2.3 on MBPP+. Reuse trails full BPTT’s pass@1 by no more than this variation on any benchmark, and truncating to Neumann-3 keeps GSM8K pass@1 within 0.35 points.

5.5 Distilled Prefill

We distill the S, M, and L learned-prior models of Section 5.2 into two-block students initialized from the teacher’s recurrent core (Section 3.2) and trained on 1/4 of the teacher’s pretraining tokens. Every prefill path is followed by the same teacher decoding at $R = 5$, on the tasks of Table 2. We compare against full teacher prefill ($R = 5$), teacher prefill stopped after three recurrences ($R = 3$) or one ($R = 1$), and the Untied 4 baseline, which prefills and decodes on its own.

Distilled prefill trades downstream accuracy for $1.5\text{--}1.8\times$ faster prefill. On 8K prompts, it trails the teacher’s downstream average by 1.0 (S), 2.3 (M), and 4.5 (L) points (Table 5). At matched latency, it edges out stopping the teacher early: it is 1–4% faster than teacher prefill at $R = 2$ and 0.5–0.9 points higher on the downstream average. Prefill gains saturate early (Figure 11): prefill at $R = 3$ stays within 1.6 points of the teacher, but it saves only 21–29% of the teacher’s latency, against 35–44% for distilled prefill. A single recurrence is 12–23% faster than distilled prefill but loses $2.7\text{--}3.5\times$ as much downstream accuracy. Better distillation objectives, student architectures such as mixture-of-experts layers, and the allocation of the distillation token budget could widen this margin and close the growing gap to the full teacher; we leave them to future work.

Teacher initialization drives the student’s quality, and the KL term raises its generation scores. Under the same hidden and KL objective, teacher initialization adds 0.7 (S) and 1.1 (M) downstream points and raises M MBPP+ from 2.50 to 6.12 (Table 22); adding the KL term raises a randomly initialized student’s M MBPP+ from 0.25 to 2.50. Appendix C.7.1 ablates the loss terms and other uses of the student, and Appendix C.7.2 distills a deeper $R = 25$ teacher.

Table 5: **Distilled prefill trades downstream accuracy for lower prefill latency.** Each path prefills the prompt, then the learned-prior teacher decodes at $R = 5$ from the path’s four terminal KV banks; Untied 4 prefills and decodes on its own. The distilled student is initialized from the teacher and trained with the hidden and KL objective of equation (4) on one quarter of its teacher’s pretraining tokens; the teacher rows prefill at $R = 5, 3, 2$, or 1. Latency: mean $\pm$ s.d. of six 8K prefills at batch size 8. See Appendix C.7.1 for evaluation details.

Prefill	PPL $\downarrow$		Downstream (%) $\uparrow$								Generation (%) $\uparrow$			Latency $\downarrow$
Path	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM	DROP	MBPP+	8K (ms)
S scale: (100M, 21.5B tokens)														
Untied 4	6.57	25.19	23.95	30.26	59.96	44.57	23.21	16.60	77.60	39.45	1.97	11.39	0.12	100.7 $\pm$ 0.5
Teacher ($R=5$)	5.67	19.91	27.36	32.51	63.00	45.50	25.09	17.20	80.90	41.65	1.59	14.08	0.62	195.0 $\pm$ 1.4
Three recurrences ($R=3$)	5.68	19.96	25.79	32.45	63.11	45.03	25.43	17.60	80.20	41.37	1.82	13.85	0.38	153.1 $\pm$ 1.7
Two recurrences ($R=2$)	5.73	20.23	22.45	31.81	62.51	43.27	24.66	16.40	79.90	40.14	1.90	13.07	0.25	131.0 $\pm$ 0.6
One recurrence ($R=1$)	5.95	21.02	19.29	31.13	61.21	41.62	23.98	15.40	73.70	38.05	1.52	10.86	0.00	110.7 $\pm$ 0.1
Distilled, teacher init	5.71	20.16	23.56	31.93	63.17	43.86	25.17	17.00	79.70	40.63	1.74	14.41	0.62	125.9 $\pm$ 1.0
M scale: (400M, 85.9B tokens)														
Untied 4	4.68	15.73	31.19	37.43	63.71	50.72	27.65	15.80	82.10	44.08	2.35	13.12	2.50	229.8 $\pm$ 1.2
Teacher ($R=5$)	3.93	12.28	38.58	46.17	67.41	57.58	28.75	17.00	88.00	49.07	15.62	17.11	10.38	495.8 $\pm$ 3.5
Three recurrences ($R=3$)	3.94	12.34	34.76	45.06	67.57	56.06	27.65	17.80	86.70	47.94	12.89	16.55	11.62	370.1 $\pm$ 1.4
Two recurrences ($R=2$)	3.99	12.53	27.34	44.04	66.05	53.87	28.07	18.20	83.80	45.91	9.17	16.22	7.00	304.2 $\pm$ 2.3
One recurrence ($R=1$)	4.23	13.12	13.76	41.60	64.53	49.45	27.90	15.40	75.70	41.19	2.27	8.56	0.12	245.9 $\pm$ 2.6
Distilled, teacher init	3.97	12.44	31.86	44.63	67.74	55.68	27.90	15.20	84.40	46.77	8.95	16.20	6.12	299.3 $\pm$ 1.6
L scale: (1.6B, 343.6B tokens)														
Untied 4	3.66	11.14	40.35	48.63	70.29	62.37	30.63	16.40	88.90	51.08	7.20	18.62	10.50	614.2 $\pm$ 3.6
Teacher ($R=5$)	3.07	8.68	55.17	61.57	73.72	71.09	40.44	24.40	93.60	60.00	47.92	34.56	27.25	1482.1 $\pm$ 32.0
Three recurrences ($R=3$)	3.08	8.70	50.15	61.01	72.80	68.69	38.99	24.60	92.50	58.39	44.28	26.98	26.38	1056.4 $\pm$ 2.4
Two recurrences ($R=2$)	3.11	8.87	39.82	58.96	71.44	64.73	35.67	24.00	90.10	54.96	33.06	20.97	20.25	848.6 $\pm$ 1.8
One recurrence ($R=1$)	3.31	9.23	19.35	53.57	70.46	58.54	31.06	20.60	82.30	47.98	6.22	12.82	0.88	642.0 $\pm$ 4.0
Distilled, teacher init	3.12	8.81	43.97	58.92	72.69	66.12	35.24	21.40	90.40	55.53	32.90	23.49	16.50	830.3 $\pm$ 3.0

6 Related Work

Looped models and fixed points. Looped language models apply a shared block repeatedly [1] to scale test-time compute [7, 42] and latent reasoning [2, 5]. As depth grows, their states approach fixed points [2, 10], which DEQs solve for directly [14]. Prior work uses these fixed points to change how many recurrences run: it stabilizes them [6, 7, 43], trains supervised states as attractors [8], or halts at convergence [44]. We further identify and verify the efficiency gains that fixed points bring to training, inference, and post-training.

Looped model efficiency. Near a fixed point, further recurrences barely change the state. We show that this property explains two shortcuts looped models already take and enables two new ones. Truncated backpropagation [2, 13] thus approximates the equilibrium gradient without the warm-up that implicit gradients need [11, 12, 45]. Of the schemes that share KV across recurrences [2, 5, 46], terminal sharing is likewise nearly lossless when training supervises near fixed points. Terminal sharing also opens a shortcut for prefill. Existing looped-model distillation keeps students recurrent (Appendix E.5), and prompt-KV predictors [21, 47] must predict each recurrence’s KV; distilled prefill instead uses a non-recurrent student to predict only the endpoint, from which one teacher recurrence builds every KV bank. Furthermore, fixed-point reuse carries endpoints across computations: it warm-starts solvers on related inputs (Appendix E.4), and we bring it to RL, scoring rollouts from saved states instead of replaying the recurrence [5].

Shaping fixed points. Two choices shape the fixed point: which depths training supervises, and how the input enters each recurrence. Randomized training depths promote path-independent equilibria [42], but existing depth distributions are fixed [2], marginalized over at inference [48], or learned as exits [49, 50]. Closest to ours, Ouro [5] and RL-Halting [51] learn entropy-regularized depth distributions from the recurrent state, but they unroll every training step to the maximum depth and do not target fixed-point shaping (Appendix E.1). Injecting the input into every recurrence makes the fixed point depend on it [2, 14, 52]. Parcae [15] improves injection by forcing contractiveness to stabilize training. We further observe that the carried-over state can have a component along the injected input; OrthoInj projects it out so every recurrence receives the input intact (Appendix E.2).

7 Discussion and Conclusion

Near a fixed point, the endpoint of a looped language model can replace its recurrent trajectory in training, inference, and post-training. Whether training produces such fixed points depends on the depth prior and on input injection. A learned depth prior lowers perplexity over a fixed PLN prior at the same mean depth while keeping terminal KV sharing intact. OrthoInj projects the carried-over state off the injected input, giving lower perplexity than the injection schemes of Huginn and Parcae. On these models, terminal KV sharing and rollout-state reuse cut memory or time at a small cost in quality, and distilled prefill trades downstream accuracy for prefill latency, with a gap that grows with scale.

Limitations. We train models of up to 1.6B parameters with one seed per configuration, and our ablations do not exhaust the design space. Larger scales and other model configurations remain untested, and the feasibility of these techniques in real-world applications needs further study.

References

- [1] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1807.03819>.
- [2] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach, 2025. URL <https://arxiv.org/abs/2502.05171>.
- [3] Zitian Gao, Yilong Chen, Yihao Xiao, Xinyu Yang, Ran Tao, Joey Zhou, and Bryan Dai. Loop the loopies!, 2026. URL <https://arxiv.org/abs/2607.16051>.
- [4] Shaowen Wang, Ge Zhang, Kairong Luo, Yuhao Wu, Shaofan Liu, Jiaheng Liu, Wenhao Huang, Shen Yan, and Jian Li. SMELT: Scaling laws for compute-matched MoE looped transformers, 2026. URL <https://arxiv.org/abs/2609.01343>.
- [5] Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, Qiyang Min, Hongzhi Huang, Xun Zhou, Wei Ye, Jiaheng Liu, Jian Yang, Yunfeng Shi, Chenghua Lin, Enduo Zhao, Tianle Cai, Ge Zhang, Wenhao Huang, Yoshua Bengio, and Jason Eshraghian. Scaling latent reasoning via looped language models, 2025. URL <https://arxiv.org/abs/2510.25741>.
- [6] Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. Stabilizing equilibrium models by Jacobian regularization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 554–565. PMLR, 2021. URL <https://proceedings.mlr.press/v139/bai21b.html>.
- [7] Xiao-Wen Yang, Ziyu Han, Xi-Hua Zhang, Wen-Da Wei, Jie-Jing Shao, Lan-Zhe Guo, and Yu-Feng Li. Stabilizing recurrent dynamics for test-time scalable latent reasoning in looped language models. *arXiv preprint arXiv:2605.26733*, 2026. URL <https://arxiv.org/abs/2605.26733>.
- [8] Benhao Huang, Zhengyang Geng, and Zico Kolter. Equilibrium reasoners: Learning attractors enables scalable reasoning. *arXiv preprint arXiv:2605.21488*, 2026.

- [9] Kristian Schwethelm, Daniel Rückert, and Georgios Kaissis. Depth-adaptive inference of looped language models via continuous depth batching, 2026. URL <https://arxiv.org/abs/2608.09444>.
- [10] Hugh Blayney, Álvaro Arroyo, Johan Obando-Ceron, Pablo Samuel Castro, Aaron Courville, Michael M. Bronstein, and Xiaowen Dong. A mechanistic analysis of looped reasoning language models, 2026. URL <https://arxiv.org/abs/2604.11791>.
- [11] Renjie Liao, Yuwen Xiong, Ethan Fetaya, Lisa Zhang, KiJung Yoon, Xaq Pitkow, Raquel Urtasun, and Richard Zemel. Reviving and improving recurrent back-propagation. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 3082–3091, 2018. URL <https://proceedings.mlr.press/v80/liao18c.html>.
- [12] Zhengyang Geng, Xin-Yu Zhang, Shaojie Bai, Yisen Wang, and Zhouchen Lin. On training implicit models. In *Advances in Neural Information Processing Systems*, 2021. URL <https://arxiv.org/abs/2111.05177>.
- [13] Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025. URL <https://arxiv.org/abs/2506.21734>.
- [14] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/01386bd6d8e091c2ab4c7c7de644d37b-Abstract.html>.
- [15] Hayden Prairie, Zachary Novack, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. Parcae: Scaling laws for stable looped language models, 2026. URL <https://arxiv.org/abs/2604.12946>.
- [16] Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. Towards looped models done right—Part I: Topology, input injection, recurrent-state design, 2026. URL https://huskydodge.github.io/husky-blog/posts/recursive_models/towards-looped-models-done-right/. Accessed 2026-07-31.
- [17] Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. Multiscale deep equilibrium models. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL <https://arxiv.org/abs/2006.08656>.
- [18] Bac Nguyen and Lukas Mauch. Efficient training of deep equilibrium models, 2023. URL <https://arxiv.org/abs/2304.11663>. ICML 2022 Workshop.
- [19] Ronald J. Williams and Jing Peng. An efficient gradient-based algorithm for on-line training of recurrent network trajectories. *Neural Computation*, 2(4):490–501, 1990. doi: 10.1162/neco.1990.2.4.490. URL <https://doi.org/10.1162/neco.1990.2.4.490>.
- [20] Christopher Aicher, Nicholas J. Foti, and Emily B. Fox. Adaptively truncating backpropagation through time to control gradient bias. In *Proceedings of The 35th Uncertainty in Artificial Intelligence Conference*, volume 115 of *Proceedings of Machine Learning Research*, pages 799–808, 2019. URL <https://proceedings.mlr.press/v115/aicher20a.html>.
- [21] Maxwell Horton, Qingqing Cao, Chenfan Sun, Yanzi Jin, Sachin Mehta, Mohammad Rastegari, and Moin Nabi. KV prediction for improved time to first token, 2024. URL <https://arxiv.org/abs/2410.08391>.
- [22] Adriana Romero, Nicolas Ballas, Samira Ebrahimi Kahou, Antoine Chassang, Carlo Gatta, and Yoshua Bengio. FitNets: Hints for thin deep nets. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6550>.
- [23] Hado P. van Hasselt, Arthur Guez, Matteo Hessel, Volodymyr Mnih, and David Silver. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL <https://arxiv.org/abs/1602.07714>.
- [24] Matteo Hessel, Hubert Soyer, Lasse Espeholt, Wojciech Czarnecki, Simon Schmitt, and Hado van Hasselt. Multi-task deep reinforcement learning with PopArt. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 3796–3803, 2019. doi: 10.1609/aaai.v33i01.33013796. URL <https://ojs.aaai.org/index.php/AAAI/article/view/4266>.
- [25] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. URL <https://arxiv.org/abs/2302.13971>.
- [26] Liping Tang, Nikhil Ranjan, Omkar Pangarkar, Xuezhi Liang, Zhen Wang, Li An, Bhaskar Rao, Linghao Jin,

- Huijuan Wang, Zhoujun Cheng, Suqi Sun, Cun Mu, Victor Miller, Xuezhe Ma, Yue Peng, Zhengzhong Liu, and Eric P. Xing. TxT360: A top-quality LLM pre-training dataset requires the perfect blend. Hugging Face dataset, 2024. URL <https://huggingface.co/datasets/LLM360/TxT360>.
- [27] Neha Sengupta, Sunil Kumar Sahu, Bokang Jia, Satheesh Katipomu, Haonan Li, Fajri Koto, William Marshall, Gurpreet Gosal, Cynthia Liu, Zhiming Chen, Osama Mohammed Afzal, Samta Kamboj, Onkar Pandit, Rahul Pal, Lalit Pradhan, Zain Muhammad Mujahid, Massa Baali, Xudong Han, Sondos Mahmoud Bsharat, Alham Fikri Aji, Zhiqiang Shen, Zhengzhong Liu, Natalia Vassilieva, Joel Hestness, Andy Hock, Andrew Feldman, Jonathan Lee, Andrew Jackson, Hector Xuguang Ren, Preslav Nakov, Timothy Baldwin, and Eric Xing. Jais and Jais-chat: Arabic-centric foundation and instruction-tuned open generative large language models. *arXiv preprint arXiv:2308.16149*, 2023. URL <https://arxiv.org/abs/2308.16149>.
- [28] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [29] Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. MiniCPM: Unveiling the potential of small language models with scalable training strategies. In *Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=3X2L2TFrOf>.
- [30] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016. URL <https://arxiv.org/abs/1609.07843>.
- [31] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1525–1534, 2016. URL <https://aclanthology.org/P16-1144/>.
- [32] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, 2019. URL <https://aclanthology.org/P19-1472/>.
- [33] Yonatan Bisk, Rowan Zellers, Ronan Le Bras, Jianfeng Gao, and Yejin Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7432–7439, 2020. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6239>.
- [34] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? Try ARC, the AI2 reasoning challenge, 2018. URL <https://arxiv.org/abs/1803.05457>.
- [35] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? A new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, 2018. URL <https://aclanthology.org/D18-1260/>.
- [36] Johannes Welbl, Nelson F. Liu, and Matt Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106, 2017. URL <https://aclanthology.org/W17-4413/>.
- [37] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.
- [38] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of NAACL-HLT*, pages 2368–2378, 2019. URL <https://aclanthology.org/N19-1246/>.
- [39] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- [40] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation, 2023. URL <https://arxiv.org/abs/2305.01210>.
- [41] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin.

- Understanding R1-Zero-like training: A critical perspective, 2025. URL <https://arxiv.org/abs/2503.20783>.
- [42] Cem Anil, Ashwini Pople, Kaiqu Liang, Johannes Treutlein, Yuhuai Wu, Shaojie Bai, J. Zico Kolter, and Roger B. Grosse. Path independent equilibrium models can better exploit test-time computation. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/331c41353b053683e17f7c88a797701d-Abstract-Conference.html.
 - [43] Shuzhen Li, Yifan Zhang, Jiacheng Guo, Quanquan Gu, and Mengdi Wang. DeepLoop: Depth scaling for looped transformers, 2026. URL <https://arxiv.org/abs/2607.13491>.
 - [44] Sajad Movahedi, Vera Milovanović, Shlomo Libo Feigin, Alexander Theus, Thomas Hofmann, Valentina Boeva, T. Konstantin Rusch, and Antonio Orvieto. Fixed-point reasoners: Stable and adaptive deep looped transformers, 2026. URL <https://arxiv.org/abs/2606.18206>.
 - [45] Samy Wu Fung, Howard Heaton, Qiuwei Li, Daniel McKenzie, Stanley Osher, and Wotao Yin. JFB: Jacobian-free backpropagation for implicit networks, 2021. URL <https://arxiv.org/abs/2103.12803>.
 - [46] Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-Recursions: Learning dynamic recursive depths for adaptive token-level computation, 2025. URL <https://arxiv.org/abs/2507.10524>.
 - [47] Aurick Qiao, Zhewei Yao, Samyam Rajbhandari, and Yuxiong He. SwiftKV: Fast prefill-optimized inference with knowledge-preserving model transformation, 2024. URL <https://arxiv.org/abs/2410.03960>.
 - [48] Javier Antorán, James Urquhart Allingham, and José Miguel Hernández-Lobato. Depth uncertainty in neural networks, 2020. URL <https://arxiv.org/abs/2006.08437>.
 - [49] Alex Graves. Adaptive computation time for recurrent neural networks, 2016. URL <https://arxiv.org/abs/1603.08983>.
 - [50] Andrea Banino, Jan Balaguer, and Charles Blundell. PonderNet: Learning to ponder, 2021. URL <https://arxiv.org/abs/2107.05407>.
 - [51] Hsun-Yu Kuo, El Mahdi Chayti, Patrik Reizinger, Wieland Brendel, and Martin Jaggi. Stabilizing extrapolation in looped transformers via learned stochastic stopping. In *Advances in Neural Information Processing Systems*, 2026. URL <https://arxiv.org/abs/2606.29983>.
 - [52] Arpit Bansal, Avi Schwarzschild, Eitan Borgnia, Zeyad Emam, Furong Huang, Micah Goldblum, and Tom Goldstein. End-to-end algorithm synthesis with recurrent networks: Extrapolation without overthinking. In *Advances in Neural Information Processing Systems*, volume 35, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/7f70331dbe58ad59d83941dfa7d975aa-Abstract-Conference.html.
 - [53] J. M. Ortega and W. C. Rheinboldt. *Iterative Solution of Nonlinear Equations in Several Variables*, volume 30 of *Classics in Applied Mathematics*. SIAM, 2000. doi: 10.1137/1.9780898719468. URL <https://epubs.siam.org/doi/book/10.1137/1.9780898719468>. Unabridged republication of the 1970 Academic Press edition.
 - [54] Sandy Research. Parcae: Official implementation, 2026. URL <https://github.com/sandyresearch/parcae>. Initialization: `parcae_lm/utils/init.py` and `parcae_lm/models/parcae/init.py`. Accessed September 15, 2026.
 - [55] Liu Yang, Kangwook Lee, Robert D. Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2311.12424>.
 - [56] Jang Hyun Cho and Bharath Hariharan. On the efficacy of knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4794–4802, 2019.
 - [57] Seyed Iman Mirzadeh, Mehrdad Farajtabar, Ang Li, Nir Levine, Akihiro Matsukawa, and Hassan Ghasemzadeh. Improved knowledge distillation via teacher assistant. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 5191–5198, 2020.
 - [58] Aakash Lahoti, Kevin Y. Li, Berlin Chen, Caitlin Wang, Aviv Bick, J. Zico Kolter, Tri Dao, and Albert Gu. Mamba-3: Improved sequence modeling using state space principles. *arXiv preprint arXiv:2603.15569*, 2026. URL <https://arxiv.org/abs/2603.15569>.

- [59] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. WinoGrande: An adversarial Winograd schema challenge at scale. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 8732–8740, 2020. URL <https://ojs.aaai.org/index.php/AAAI/article/view/6399>.
- [60] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. A framework for few-shot language model evaluation, 2023. URL <https://zenodo.org/records/10256836>.
- [61] Andrei Cristian Popescu, Haitz Sáez de Ocáriz Borde, and Pietro Liò. Adaptive depth in looped transformers: Diagnosing learned halting gates and trajectory readouts, 2026. URL <https://arxiv.org/abs/2607.20519>.
- [62] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Trellis networks for sequence modeling. In *International Conference on Learning Representations*, 2019. URL <https://arxiv.org/abs/1810.06682>.
- [63] Giyeong Oh, Woohyun Cho, Siyeol Kim, Suhwan Choi, and Youngjae Yu. Revisiting residual connections: Orthogonal updates for stable and efficient deep networks, 2025. URL <https://arxiv.org/abs/2505.11881>.
- [64] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- [65] Victor Conchello Vendrell, Arnau Padrés Masdemont, Niccolò Grillo, Jordi Ros-Giralt, Arash Behboodi, and Fabio Valerio Massoli. Memory-efficient looped transformer: Decoupling compute from memory in looped language models, 2026. URL <https://arxiv.org/abs/2605.07721>.
- [66] Shaojie Bai, Zhengyang Geng, Yash Savani, and J. Zico Kolter. Deep equilibrium optical flow estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 620–630, 2022. URL <https://arxiv.org/abs/2204.08442>.
- [67] Can Ufuk Ertenli, Emre Akbas, and Ramazan Gokberk Cinbis. Streaming multiscale deep equilibrium models. In *European Conference on Computer Vision*, 2022. doi: 10.1007/978-3-031-20083-0_12.
- [68] Xingjian Bai and Luke Melas-Kyriazi. Fixed point diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9430–9440, 2024. URL <https://arxiv.org/abs/2401.08741>.
- [69] Sangmin Bae, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Seungyeon Kim, and Tal Schuster. Relaxed recursive transformers: Effective parameter sharing with layer-wise LoRA. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2410.20672>.
- [70] James O’Neill and Fergal Reid. Looped latent attention: Cross-loop KV compression for looped transformers, 2026. URL <https://arxiv.org/abs/2607.15456>.
- [71] Ahmadreza Jeddi, Marco Ciccone, and Babak Taati. LoopFormer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *International Conference on Learning Representations*, 2026. URL <https://arxiv.org/abs/2602.11451>.
- [72] Shaojie Bai, Vladlen Koltun, and J. Zico Kolter. Neural deep equilibrium solvers. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=B0oH0wT5ENL>.

Appendix Contents

Appendix A gives the theoretical analysis, including the proof of Lemma 1. Appendix B specifies the methods and training configurations, including why depth sampling exposes the model to converged context. Appendix C collects supplementary experiments, including rollout-state reuse in RL, and Appendix D defines the shared datasets and evaluation protocols. Appendix E relates the learned depth prior, OrthoInj, terminal KV sharing, rollout-state reuse, and distilled prefill to prior work.

A Theoretical Analysis and Proofs	20
A.1 Joint Iteration and Frozen-Prefix Decoding	20
A.2 Truncated Backpropagation and Implicit Gradients	21
B Additional Method Details	23
B.1 Models and Training Contract	23
B.2 Why Sample Training Depths?	24
B.3 Learned Depth Prior	25
B.4 Architecture Variants and Initialization	26
C Additional Experimental Results	27
C.1 Depth-Prior Learning and Test-Time Scaling	27
C.2 Input Injection	31
C.3 Truncation, Depth Sampling, and Fixed Points	33
C.4 Recurrent-State and Cache Dynamics	34
C.5 Terminal KV Sharing at Deeper Recurrence	36
C.6 Independent Prefill and Decode Budgets	36
C.7 Distilled Prefill	37
C.8 Reusing States during Post-Training	40
D Datasets and Evaluation Protocols	43
D.1 Mixture-Matched Validation	43
D.2 WikiText-103 and Zero-Shot Tasks	43
D.3 Terminal-KV Evaluation	44
D.4 Cached-Continuation Perplexity	45
D.5 Generation Benchmarks	45
E Extended Related Work	45
E.1 Learned Depth Distributions and Exits	45
E.2 Input Injection and Orthogonal Updates	46
E.3 KV Sharing Across Recurrent Depth	46
E.4 Fixed-Point Reuse	47
E.5 Distillation for Looped Models	47

A Theoretical Analysis and Proofs

We prove Lemma 1 and extend it to token-by-token decoding and finite depth (Appendix A.1), and relate truncated backpropagation to implicit gradients (Appendix A.2).

A.1 Joint Iteration and Frozen-Prefix Decoding

Fix parameters, tokens, injection, positions, and masks. As in Section 3.2, h includes all current-token recurrent variables and C all prefix inputs, including per-layer KV and additional recurrent state. If C contains inputs other than KV, sharing only the KV banks falls under Lemma 1 only when those other inputs also coincide. By the triangle inequality, the assumptions of Lemma 1 give, for all $h, \tilde{h} \in \mathcal{H}$ and $C, \tilde{C} \in \mathcal{C}$, the joint Lipschitz bound

$$\|f_\theta(h; C) - f_\theta(\tilde{h}; \tilde{C})\| \leq \kappa \|h - \tilde{h}\| + \beta \|C - \tilde{C}\|. \quad (9)$$

Proof of Lemma 1. By the contraction mapping theorem [53], $f_\theta(\cdot; C_{<t}^*)$ has a unique fixed point h^* in the complete space $\mathcal{H}$, and $\tilde{h}^r \rightarrow h^*$. Let $\delta_r = \|h^r - h^*\|$. Invariance keeps $h^r \in \mathcal{H}$, so equation (9) and $h^* = f_\theta(h^*; C_{<t}^*)$ give

$$\delta_{r+1} \leq \kappa \delta_r + \beta \|C_{<t}^r - C_{<t}^*\|. \quad (10)$$

With $\eta_{\max} = \sup_r \|C_{<t}^r - C_{<t}^*\| < \infty$, induction gives $\delta_r \leq \max\{\delta_0, \beta \eta_{\max}/(1 - \kappa)\}$, so $\bar{\delta} = \limsup_r \delta_r$ is finite, and taking $\limsup$ in equation (10) gives $\bar{\delta} \leq \kappa \bar{\delta}$. Hence $\bar{\delta} = 0$ and $h^r \rightarrow h^*$. Both iterations thus reach h^* and $C_{<t}^*$, so any readout continuous in both has the same limit. $\square$

Extension to one-by-one decoding. For N tokens and nonempty closed sets $\mathcal{H}_t \subseteq \mathbb{R}^{d_t}$, write

$$C_{<t}^r = \Phi_{\theta,t}(h_{<t}^r), \quad h_t^{r+1} = f_{\theta,t}(h_t^r; C_{<t}^r), \quad (11)$$

where $h_{<t}^r = (h_1^r, \dots, h_{t-1}^r)$ and $C_{<1}^r = C_{<1}$. Each $\Phi_{\theta,t}$ is continuous and depth-shared, and for every context $C \in \Phi_{\theta,t}(\prod_{t' < t} \mathcal{H}_{t'})$, $f_{\theta,t}(\cdot; C)$ maps $\mathcal{H}_t$ into itself; on these states and contexts, $f_{\theta,t}$ satisfies equation (9) with constants $\kappa_t < 1$ and $\beta_t < \infty$. From initial states in these sets, joint iteration and sequential decoding, which solves and freezes each token in turn, converge to the same limiting states; with common continuous readouts, they also give the same predictions.

Proof. Induct on t . At $t = 1$ both executions use the fixed context $C_{<1}$ and converge to the fixed point h_1^* of $f_{\theta,1}(\cdot; C_{<1})$. If $h_{t'}^r \rightarrow h_{t'}^*$ for all $t' < t$, continuity gives $C_{<t}^r \rightarrow \Phi_{\theta,t}(h_{<t}^*) =: C_{<t}^*$, the context that sequential decoding freezes at position t . Invariance and closedness keep $C_{<t}^r$ and $C_{<t}^*$ in $\Phi_{\theta,t}(\prod_{t' < t} \mathcal{H}_{t'})$, so Lemma 1 gives $h_t^r \rightarrow h_t^*$. Continuity of the readouts completes the proof, whatever the order in which joint positions converge. $\square$

Finite depth and approximate terminal context. Let $\eta = \|\bar{C}_{<t} - C_{<t}^*\|$ and $\bar{h}^{r+1} = f_\theta(\bar{h}^r; \bar{C}_{<t})$, and assume that the invariance and Lipschitz conditions also hold for $\bar{C}_{<t}$. Equation (10) with $h^r, C_{<t}^r$ replaced by $\bar{h}^r, \bar{C}_{<t}$ unrolls to

$$\|\bar{h}^R - h^*\| \leq \kappa^R \|\bar{h}^0 - h^*\| + \frac{\beta(1 - \kappa^R)}{1 - \kappa} \eta. \quad (12)$$

The two terms capture unfinished token refinement and imperfect context, respectively; the second bounds the fixed-point displacement by $\beta\eta/(1 - \kappa)$. A readout or loss with state and context Lipschitz constants L_h and L_C differs by at most L_h times the right-hand side of equation (12) plus $L_C\eta$. Here η includes errors in earlier decoded tokens' banks.

Empirical corroboration. We compare joint iteration with current-token iteration after solving and freezing the prefix, using the matched Fixed $R = 5$ and independently sampled PLN-5 Small Hugginn checkpoints of Table 2. On prefixes passing the convergence checks, the two paths reach nearly identical endpoints within each model (Figure 6), consistent with Lemma 1. Freezing a depth-five prefix instead leaves a gap five to six orders of magnitude larger: the median relative endpoint gap is 0.27 for Fixed $R = 5$ and 0.033 for PLN-5, against about 4×10^{-7} with a converged prefix. This finite-prefix control shows that the lemma needs the converged context. Qualified prefixes need about a hundred or more recurrences to converge, far beyond the depth five used for sharing, so practical sharing rests on the finite-depth bound in equation (12) and on the measured cached perplexity (Table 2), not on exact convergence.

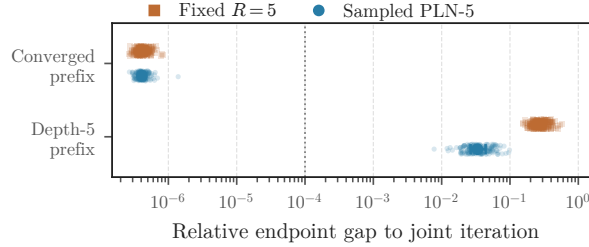


Figure 6: **Freezing a converged prefix reproduces joint iteration (Lemma 1).** Relative gap between the current-token endpoint of joint iteration and of iteration with the prefix frozen after convergence or after five recurrences. Dots are prefixes that pass the convergence checks (206 of 304 for Fixed $R = 5$, 191 of 304 for PLN-5); markers and bars give medians and interquartile ranges. The dotted line marks the 10^{-4} agreement tolerance.

We use 304 spans, 16 from each validation source, with 1,534 prefix tokens and one current token. Prefix-only and joint solves start independently from the model’s evaluation-time state initialization. The frozen-prefix path updates only the current token, retaining each physical layer’s historical KV and recomputing its own KV. All solves use FP32, PyTorch’s reference (math) attention kernel, and at most 1,024 recurrences. A solve qualifies when the maximum per-token relative state change and per-layer K/V change remain at most 10^{-5} for eight steps, followed by 16 confirmation steps with state drift at most 5×10^{-5} and a final one-step check. This criterion rules out cycles.

All three solves (prefix-only, joint, and frozen-prefix current-token) qualify on 206 Fixed $R = 5$ and 191 PLN-5 spans; the other 98 and 113 spans fail these checks and are dropped. Qualified prefix solves need median depths of 152.5 (Fixed $R = 5$) and 114 (PLN-5), with maxima of 890 and 961. Every qualified pair agrees within relative state error 10^{-4} and prediction KL 10^{-7} ; the observed maximum state gaps are 8.24×10^{-7} and 1.39×10^{-6} . The same checks pass from zero current-token initialization. The depth-five control uses the same qualified spans and requires its own current-token solve to pass. The two models’ qualifying sets need not coincide.

A.2 Truncated Backpropagation and Implicit Gradients

For fixed input x , complete recurrent state $z = (\mathbf{H}, C) \in \mathbb{R}^{d_z}$ of Section 2, and parameters $\theta \in \mathbb{R}^{d_\theta}$, we connect equation (1) to backpropagation through the final recurrent steps.

Finite-depth and equilibrium objectives. For a training-depth distribution p , the sampled-depth and equilibrium losses optimize different predictions:

$$\mathcal{L}_{\text{depth}} = \mathbb{E}_{R \sim p}[\ell_\theta(z^R, y)], \quad \mathcal{L}_{\text{eq}} = \mathbb{E}[\ell_\theta(z^\star, y)]. \quad (13)$$

Full BPTT differentiates the finite trajectory; implicit differentiation differentiates a local equilibrium branch. Equal states need not have equal derivatives. For $F_{\theta,a}(z; x) = az + (1-a)e$ with the prelude output $e = P_{\theta}(x)$, $z^0 = 0$, and $0 \leq a < 1$, the equilibrium $z^* = e$ is independent of a , whereas $z^R = (1 - a^R)e$ depends on it. Where $e = 0$, both states are zero, but $dz^R/d\theta = (1 - a^R) de/d\theta$ differs from $dz^*/d\theta = de/d\theta$. TBPTT truncates the trajectory gradient; a convergent Neumann series approximates equilibrium sensitivity. Before convergence, the Neumann estimate is only a surrogate, so comparisons must state the objective and backward budget.

Equilibrium sensitivity and loss gradient. At $z^* = F_{\theta}(z^*; x)$ [14], define $J_{\star} = \partial_z F_{\theta}(z^*; x) \in \mathbb{R}^{d_z \times d_z}$ and $B_{\star} = \partial_{\theta} F_{\theta}(z^*; x) \in \mathbb{R}^{d_z \times d_{\theta}}$, holding the state fixed in $B_{\star}$. Continuous differentiability near the equilibrium and invertibility of $I - J_{\star}$ give a locally differentiable equilibrium branch by the implicit function theorem:

$$\underbrace{\frac{dz^*}{d\theta}}_{D_{\star}} = \underbrace{\partial_z F_{\theta}}_{J_{\star}} \frac{dz^*}{d\theta} + \underbrace{\partial_{\theta} F_{\theta}}_{B_{\star}}, \quad (I - J_{\star})D_{\star} = B_{\star}. \quad (14)$$

Thus $D_{\star} = (I - J_{\star})^{-1}B_{\star}$. For a single example, $\mathcal{L}_{\text{eq}}(\theta) = \ell_{\theta}(z^*, y)$; define column gradients $g = \nabla_z \ell_{\theta}$ and $c = \nabla_{\theta} \ell_{\theta}$ at the equilibrium, holding the state fixed in c . The loss gradient is $G_{\text{imp}} = c + D_{\star}^{\top}g$. Its adjoint form,

$$(I - J_{\star}^{\top})v = g, \quad G_{\text{imp}} = c + B_{\star}^{\top}v, \quad (15)$$

uses the vector–Jacobian products (VJPs) $J_{\star}^{\top}v$ and $B_{\star}^{\top}v$ through one evaluation of F_{θ} , with v fixed, avoiding an explicit Jacobian or stored forward trajectory [14].

Neumann truncation and a detached suffix. The stronger condition that $J_{\star}$ has spectral radius below one ensures $J_{\star}^b \rightarrow 0$, so the identity $(I - J_{\star}) \sum_{k=0}^{b-1} J_{\star}^k = I - J_{\star}^b$ yields

$$D_{\star} = \sum_{k=0}^{\infty} J_{\star}^k B_{\star}, \quad D_b = \sum_{k=0}^{b-1} J_{\star}^k B_{\star}, \quad D_{\star} - D_b = J_{\star}^b D_{\star}. \quad (16)$$

Equivalently, the iteration $v_0 = 0$, $v_{k+1} = g + J_{\star}^{\top}v_k$ gives $G_b = c + B_{\star}^{\top}v_b$ [11, 12]. Invertibility suffices for equation (15); the spectral condition ensures convergence.

Starting from $\bar{z}^0 = \text{stopgrad}(z^*)$, each update $\bar{z}^{k+1} = F_{\theta}(\bar{z}^k; x)$ remains at the equilibrium and uses the same $J_{\star}, B_{\star}$. Its sensitivity after k updates is D_k , since $D_0 = 0$ and $D_{k+1} = J_{\star} D_k + B_{\star}$; hence the loss gradient is $c + D_b^{\top}g = G_b$. A Euclidean contraction bound controls the omitted tail:

Lemma 2 (Truncated backpropagation at equilibrium; adapted from Liao et al. [11]). *Suppose F_{θ} is continuously differentiable near the current equilibrium, ℓ_{θ} is differentiable there, and $\|J_{\star}\|_2 \leq \kappa < 1$. Start from $\text{stopgrad}(z^*)$ and differentiate b applications of the same deterministic recurrence, retaining all direct parameter paths. Then*

$$G_b = c + B_{\star}^{\top} \sum_{k=0}^{b-1} (J_{\star}^{\top})^k g, \quad \|G_{\text{imp}} - G_b\|_2 \leq \|B_{\star}\|_2 \|g\|_2 \frac{\kappa^b}{1 - \kappa}. \quad (17)$$

Proof of Lemma 2. The sensitivity recursion above establishes the expression for G_b . Subtracting it from the infinite series and using $\|J_{\star}\|_2 \leq \kappa < 1$ gives

$$\|G_{\text{imp}} - G_b\|_2 = \left\| B_{\star}^{\top} \sum_{k=b}^{\infty} (J_{\star}^{\top})^k g \right\|_2 \leq \|B_{\star}\|_2 \|g\|_2 \sum_{k=b}^{\infty} \kappa^k = \|B_{\star}\|_2 \|g\|_2 \frac{\kappa^b}{1 - \kappa}.$$

□

This specializes the TBPTT/Neumann connection [11, 12]. If only the spectral radius of $J_\star$ is below one, then for any γ between the spectral radius and one, $\|J_\star^k\|_2 = O(\gamma^k)$, and the same argument gives $\|G_{\text{imp}} - G_b\|_2 = O(\|B_\star\|_2 \|g\|_2 \gamma^b)$. Direct paths remain essential: $B_\star$ includes input/prelude injection, and c includes the output head. Detaching prefix KV removes further dependencies and therefore changes the estimator.

Finite-trajectory BPTT and TBPTT. Along $z^{r+1} = F_\theta(z^r; x)$, let $J_r = \partial_z F_\theta(z^r; x)$ and $B_r = \partial_\theta F_\theta(z^r; x)$. The recursion $\partial z^{r+1}/\partial\theta = J_r \partial z^r/\partial\theta + B_r$ from a detached window start z^{R-b} gives $\partial z^R/\partial\theta = \tilde{D}_b$, the TBPTT sensitivity of equation (1) with $i = R - 1 - k$:

$$\tilde{D}_b = \sum_{i=R-b}^{R-1} (J_{R-1} J_{R-2} \cdots J_{i+1}) B_i. \quad (18)$$

The empty product is I . Full BPTT includes all terms $i = 0, \dots, R-1$, which give $\tilde{D}_R$, and $(J_{R-1} \cdots J_0) dz^0/d\theta$ for parameter-dependent initialization. Multiplying the transposed sensitivity by the endpoint loss gradient and adding the loss’s direct parameter gradient gives the loss gradient.

Equation (18) reduces to D_b when all suffix Jacobians equal $J_\star$, $B_\star$ and approaches the endpoint Neumann estimate as they converge. Because early derivatives come from transient states and truncation adds its own error, neither estimator is uniformly more accurate. Controlling the error at approximate states additionally requires locally Lipschitz state and parameter Jacobians, a uniform state-Jacobian bound below one, regular loss derivatives, and small state errors throughout the window. A small endpoint residual alone is not enough.

B Additional Method Details

We specify the shared model and training configuration (Appendix B.1), explain why sampled training depths expose the model to converged context (Appendix B.2), and detail the learned depth prior of Section 4.1 (Appendix B.3), and the input-injection recipes of Section 4.2 (Appendix B.4).

B.1 Models and Training Contract

The model layout follows Section 5.1, and Table 6 gives the full configurations. All models use separate input and output embeddings. They use the Jais64k tokenizer [27] with a 64,256-token vocabulary and bfloat16 arithmetic. The weighted pretraining mixture covers web text, QA, code, mathematics, reasoning, planning, multilingual text, and synthetic data. Its QA component includes TxT360 [26]. Architectural variants and Untied 12 differ in size, so each comparison states its scale and configuration.

Each microbatch draws one training depth (Section 4.1), so a sampled-depth run draws one depth per gradient-accumulation step. Compute depends on the realized depths and backward truncation.

Learning-rate selection. Each scale has one learning-rate range, and every run tunes its peak rate within it: $[0.5, 4] \times 10^{-3}$ at S, $[0.25, 4] \times 10^{-3}$ at M, and $[0.125, 2] \times 10^{-3}$ at L. Screens keep the final WSD horizon but stop after 512–1,024 updates at S, 2,048–12,288 at M, and 8,192–32,768 at L, and select by native validation CE at $R = 5$ on 304 frozen 1,536-token sequences, distinct from MixVal. Every selected rate is interior to its run’s grid. A variant that changes only λ_H or the depth-draw correlation reuses the rate of its screened configuration. M OrthoInj in Table 3 instead trains at 10^{-3} , Parcae-Decay’s selected M rate; its own screen selects 1.5×10^{-3} (Appendix C.2 compares both rates).

Table 6: **Model configurations at three scales.** Blocks count prelude, recurrent-core, and coda blocks of the looped models; Untied 4 and Untied 12 stack 4 and 12 blocks. Width and FFN give hidden and feedforward dimensions, and Q and KV give query and key-value head counts. Non-emb. and Total give Untied 4 parameter counts without and with embeddings; the looped rows add Huginn’s width×width input adapter. The main text rounds the scales to 100M, 400M, and 1.6B. Parameter and token counts are in billions; the peak learning rate is tuned per run (Appendix B.1).

	S	M	L
A: Architecture and attention geometry			
Blocks (prelude/core/coda)	1 / 2 / 1		
Width	1,536	3,072	6,144
FFN	4,352	8,704	17,408
Q heads	24	48	96
KV heads	6	12	24
Head dim.	64	64	64
B: Parameters (B)			
Non-emb.	0.104	0.415	1.661
Total	0.301	0.810	2.451
Looped non-emb.	0.106	0.425	1.699
Looped total	0.304	0.819	2.488
C: Batch and token budget			
Sequence length	8,192		
Global batch (sequences)	512		
Tokens per update	4,194,304		
Updates	5,120	20,480	81,920
Tokens (B)	21.47	85.90	343.60
D: Optimizer			
Optimizer	AdamW		
(β_1, β_2)	(0.90, 0.98)		
ϵ	10^{-8}		
Weight decay	0.10		
Gradient clipping	1.00		
LR schedule	WSD		
Linear warmup	first 5% of updates		
Cosine decay	last 10%, to $0.1 \times$ peak		

B.2 Why Sample Training Depths?

Let $p(R)$ be the training-depth distribution and ℓ_t the next-token CE at position t . Sampling one sequence-wide horizon optimizes $\mathcal{L}_{\text{depth}}$ of equation (13) with $\ell_\theta(z^R, y) = N^{-1} \sum_t \ell_t$ [2]. Fix parameters, input, initialization, and a trajectory whose prefix KV is converged from depth r_c . Backpropagating through the final $1 \leq b(R) \leq R$ steps gives the expected number of backpropagated updates that read the converged prefix KV, which we call its exposure,

$$w_{r_c} = \sum_R p(R) \min\{b(R), (R - r_c)_+\}, \quad (\cdot)_+ = \max(\cdot, 0). \quad (19)$$

The window $R - b(R) + 1, \dots, R$ intersects updates after r_c in $\min\{b(R), (R - r_c)_+\}$ steps. Equation (19) gives $w_{r_c} = 0$ for fixed $R = 5$ when $r_c \geq 5$, but positive PLN-5 exposure whenever $p(R > r_c) > 0$; sufficiently deep fixed training can also provide exposure. With our rule $b(R) = \min(R, 10)$ (Appendix B.3) and the depths drawn by the Small PLN-5 run of Table 2, PLN-5 gives $w_5 = 1.10$ ($p(R > 5) = 35.4\%$) and $w_{10} = 0.15$, so exposure concentrates on prefixes that converge within about ten recurrences. The count ignores attention weights and gradient magnitudes, and Figure 2 does not give per-bank convergence

depths or training-time read traces.

Random-depth loss alone does not imply KV stationarity: $z^r = (z_a, (-1)^r z_b)$ has constant loss if the head reads only z_a , while any KV that reads the second component alternates. Supervising late converged states also does not ensure that early, transient states respond well to terminal KV. Exposure is thus necessary but not sufficient for sharing, so Table 2 measures sharing directly.

B.3 Learned Depth Prior

Every learned prior uses equation (6). Table 11’s four reward arms (CE, log CE, exp CE, and inverse perplexity, $-\exp(-\text{CE})$) use $\lambda_H = 0.01$, and Table 2 reports inverse perplexity at $\lambda_H \in \{0, 0.01\}$.

Sampling and controller updates. The fixed PLN-5 baseline caps equation (5) at $R \leq 64$ and recalibrates μ so that $\mathbb{E}[\min(R, 64)] = 5$. The learned prior initializes from this capped PLN: $\phi_0 = \log p_{\text{PLN}}$ uses 64 logits; tail mass accumulates at depth 64 and all masses are positive. Sampling and update order follow Algorithm 2.

The backpropagation cap is $b_{\max} = 10$; earlier recurrent states are detached, but gradients still reach the prelude through the input injected at each backpropagated recurrence, and the output head. Equation (6) uses $\bar{R} = 5$ and $\lambda_m = 1$; global token weighting is specified below. The prior’s separate Adam optimizer uses LR 10^{-3} , betas (0.90, 0.999), and zero weight decay. During a prior warmup of 32 updates, the controller collects its baseline without changing the logits. The fixed arm freezes the logits in the same sampler.

Loss aggregation and baseline. For microbatch i , detached loss sums and unmasked target counts are reduced across ranks, and dividing by the global count N_i gives ℓ_i . The within-step average of a per-microbatch statistic χ_i is

$$\hat{\mathbb{E}}_n[\chi] = \frac{\sum_i N_i \chi_i}{\sum_i N_i}. \quad (20)$$

For raw-CE control, with ℓ the globally reduced mean CE at sampled depth r , $\hat{\ell}_n$ the pre-step baseline, and $\bar{R}$ the target mean, the controller minimizes

$$\mathcal{J}_{\text{CE}}(\phi) = \hat{\mathbb{E}}_n \left[(\text{stopgrad}(\ell) - \hat{\ell}_n) \log p_\phi(r) \right] - \lambda_H H(p_\phi) + \lambda_m (\mathbb{E}_{p_\phi}[R] - \bar{R})^2. \quad (21)$$

Both regularizers are evaluated analytically over active depths and differentiated only through the logits. The objective and baseline share this weighting. The first weighted CE initializes the baseline, and its EMA also runs during the prior warmup:

$$\hat{\ell}_{n+1} = \nu \hat{\ell}_n + (1 - \nu) \hat{\mathbb{E}}_n[\text{stopgrad}(\ell)], \quad (22)$$

where $\nu \in [0, 1)$ is the EMA decay, set to 0.95.

Alternative prior signals. Three PLN-initialized arms replace raw CE in equation (6) by one of three rewards, computed after global token-weighted reduction: $\omega(\ell) = -\log(\max(\ell, 10^{-12}))$, $-\exp(\ell)$, or $\exp(-\ell)$. Their advantage is $A_i = s_{n-1}(u_i - \hat{u}_{n-1})$ with $u_i = \omega(\ell_i)$, where s is the ratio of previous CE and reward standard deviations, each floored at 10^{-12} ; it starts at one. These arms retain the model CE objective and mean penalty, adding no forward passes.

Reward form and entropy. Table 11 names each arm by its cost $-\omega(\ell)$. Centering, scaling, and optimization order follow Algorithm 2. The entropy in Section 4.1 has logit derivative $\partial H / \partial \phi(r) = -p_\phi(r)[\log p_\phi(r) + H(p_\phi)]$, requiring no model forward pass. It does not constrain tail mass at a particular depth; Appendix C.1 evaluates its coefficient.

Uniform controls. $U_{1:64}$ initializes all logits equally (mean 32.50) and retains full support. $U_{1:10}$ restricts uniform initialization and normalization to depths 1–10 (mean 5.50), keeping depths 11–64 inactive. Both target a mean depth of five and use raw-CE feedback with the PLN arm’s model and prior optimizers. The restricted control changes both initialization and support. Within the training budget, the mean penalty does not pull $U_{1:64}$ down to five: its final prior mean is 6.15 (Table 11).

Compute accounting. Backward-window visits add backward and recomputation work. Model-FLOP totals count each logical update once and exclude recomputation, controller work, and repeated work after restarts. Learning rates, schedules, and batch layouts follow Appendix B.1.

B.4 Architecture Variants and Initialization

Table 7 defines the recipes of Section 4.2; OrthoInj’s anchor is the vector it projects against.

Table 7: **Input-injection recipes.** $e = P_\theta(x)$ is the prelude output, $\tilde{e}$ is the injected input, and RMSNorm uses learned gains. DEQ-QKV injects at internal sites (superscript (j) indexes a recurrent layer); the others inject at the core entrance. $\Lambda = \text{diag}(\exp(-\Delta \odot \alpha))$ with $\Delta = \text{softplus}(\Delta_{\text{raw}})$ and $\alpha = \exp(\alpha_{\text{raw}})$, and OrthoInj applies Q_q of equation (8) with $q = \Delta \odot W\tilde{e}$ and $\epsilon = 10^{-6}$.

Injection recipe	Injected input $\tilde{e}$	Input to recurrent core
DEQ-QKV	e	$W_{\text{QKV}}^{(j)} h^{r,(j)} + W_e^{(j)} \tilde{e}$
Huginn-Linear [2]	e	$W_{\text{lin}}[h^r; \tilde{e}; 1]$
Parcae-Decay [15]	$\text{RMSNorm}(e)$	$\Lambda h^r + \Delta \odot W\tilde{e}$
Parcae w/o norm	e	$\Lambda h^r + \Delta \odot W\tilde{e}$
OrthoInj (ours)	e	$Q_q \Lambda h^r + q$

Input source and retention. DEQ-QKV injects the prelude output, which stays fixed across recurrences, into Q/K/V through separate bias-free, Gaussian-initialized projections in the two physical recurrent layers, shared across visits. It replaces the injection at the core entrance and the diagonal retention of earlier states, and keeps the prelude and the Gaussian initialization of the recurrent state.

Huginn-Linear uses a learned concatenation projection (the appended 1 gives its bias) followed by a parameter-free RMSNorm after the full core:

$$\zeta_t^r = W_{\text{lin}}[h_t^r; e_t; 1], \quad \mathbf{H}^{r+1} = \text{RMSNorm}_{\text{exit}}[\Gamma_\theta(\zeta_1^r, \dots, \zeta_N^r)]. \quad (23)$$

The diagonal injection recipes use

$$\begin{aligned} \Delta &= \text{softplus}(\Delta_{\text{raw}}), & \alpha &= \exp(\alpha_{\text{raw}}), & \Lambda &= \text{diag}(\exp(-\Delta \odot \alpha)), \\ q_t &= \Delta \odot W\tilde{e}_t, & \zeta_t^r &= \Lambda h_t^r + q_t. \end{aligned} \quad (24)$$

Here $\Delta_{\text{raw}}, \alpha_{\text{raw}} \in \mathbb{R}^d$ are shared across visits, $\odot$ denotes channelwise multiplication, $W \in \mathbb{R}^{d \times d}$ is a full, bias-free input projection, and q_t is the injected vector of Section 4.2; the core update $\mathbf{H}^{r+1} = \Gamma_\theta(\zeta_1^r, \dots, \zeta_N^r)$ follows equation (7). With the prelude output $e = P_\theta(x)$ of Section 2, Parcae-Decay uses $\tilde{e} = \text{RMSNorm}(e)$ with learned gains, $W_{\text{init}} = I$, $\alpha_{\text{init}} = 1$, and $\Delta_{\text{init}} = \frac{1}{2} \log 5$ (implemented by inverse softplus). This gives retention $1/\sqrt{5}$, matching the intended linear-injection scale [54]; prelude normalization addresses large-output instability [15, Appendix J]. Parcae w/o norm uses $\tilde{e} = e$ and $\Delta_{\text{init}} = 1$, retaining $W_{\text{init}} = I$ and $\alpha_{\text{init}} = 1$; it changes both normalization and initialization. OrthoInj uses $\tilde{e} = e$ without prelude RMSNorm and the Parcae-Decay initialization ($W_{\text{init}} = I$, $\alpha_{\text{init}} = 1$, $\Delta_{\text{init}} = \frac{1}{2} \log 5$). It projects the carryover against the injected vector q as in Section 4.2, with FP32 projection inner products and $\epsilon = 10^{-6}$, and retains all block RMSNorms. An earlier OrthoInj variant, used only for the M learning-rate screen (Appendix B.1),

uses $\Delta_{\text{init}} = 1$ and the raw input e as its anchor. Unlike the released Parcae implementation, which applies a learned linear readout C to the final recurrent state before the coda [54], these diagonal-injection recipes pass the final state to the coda directly ($C = I$); the readout lies outside the recurrence and does not enter the fixed-point iteration.

Linear injection and recurrent normalization. The original Huginn normalizes before attention and FFN, after residual additions, and at the core exit [2]. We use PreNorm blocks; Huginn-Linear adds the parameter-free core-exit RMSNorm in equation (23).

Without exit normalization, S PLN-25 training reached a nonfinite loss at all five tested learning rates. Replaying a fixed checkpoint traced the amplification to the injection interface; adding the exit RMSNorm kept the same inputs finite through recurrence 64. This replay only diagnoses forward stability; every evaluated Huginn-Linear model is trained from scratch with the exit norm.

C Additional Experimental Results

We report supplementary results and their settings; Appendix D gives the shared protocols.

C.1 Depth-Prior Learning and Test-Time Scaling

Training and evaluation. Every depth-prior run uses Parcae w/o norm injection (Table 7). Every sampled-depth row of Table 2 draws one depth per microbatch, independently across microbatches and shared across ranks. Runs use one training seed and the setup of Appendix B.1.

Native MixVal and CacheEval follow Appendix D.1 and Appendix D.4 at depth five. Sharing penalties are within-checkpoint ratios $100(\text{PPL}_{\text{terminal}}/\text{PPL}_{\text{FullKV}} - 1)$, not differences from native MixVal. Model-FLOP accounting follows Appendix B.3. For Fixed PLN-5, the trainer logs a constant nominal cost per update, so Tables 2 and 11 instead apply the learned priors’ per-depth cost to the depths each run drew. Every column of Table 2 except FLOPs follows the terminal-KV protocol of Appendix D.3, and Table 8 evaluates the same checkpoints without KV reuse. The task AVG excludes the generation scores (GSM8K, DROP, and MBPP+).

Matched M entropy-coefficient ablation. Figure 7 varies $\lambda_H \in \{0, 0.01, 0.05, 0.1\}$ at M scale. As λ_H rises, the mean depth stays near five while $p_\phi(5)$ falls from 67.4% to 14.0% (Table 10). WikiText PPL at $R = 5$ rises from 12.89 to 13.20, and at $R = 32$, $\lambda_H = 0.01$ and 0.05 tie for the lowest PPL (12.93; panel (a)). Panel (b) isolates the cost of KV sharing by scoring each checkpoint with per-visit and shared KV. Sharing raises the continuation PPL on WikiText (after a 1,024-token prefill, Appendix D.2) of fixed-depth training from 12.19 to 16.71, whereas PLN-5 and all learned priors retain or slightly improve it. On WikiText and LAMBADA, higher entropy does not monotonically improve sharing. Without KV reuse (Table 8), $\lambda_H = 0$ gives slightly lower validation PPL than $\lambda_H = 0.01$ at M and L (4.55 vs. 4.57 and 3.53 vs. 3.54) and ties it at S, but with terminal KV sharing it falls behind at L (Table 2): GSM8K 36.47 vs. 47.92.

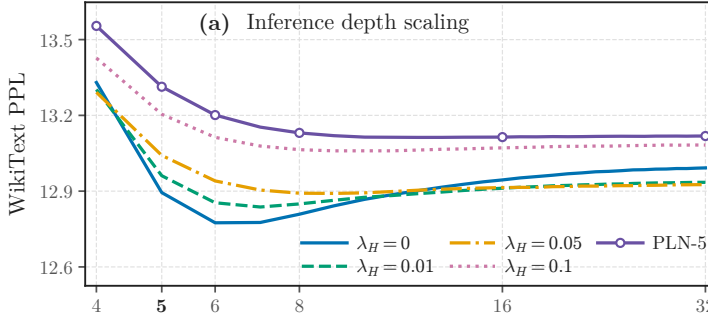
These M runs differ only in λ_H : they share model, data, seed, learning-rate schedule, and the controller of Appendix B.3 (prior LR 0.001, $\lambda_m = 1$).

Paired readouts follow Appendix D.2 and Appendix D.4; paired LAMBADA last-word PPL averages over the 5,153 words and includes the prefill prediction. Native seven-task averages, with WinoGrande instead of SciQ (Appendix D.2), stay within 43.7–44.0 across λ_H .

Paired task accuracy with and without KV sharing. Table 9 reports paired FullKV and terminal task scoring for the learned priors at $R_{\text{pre}} = R_{\text{dec}} = 5$. Sharing does not lower the average of any learned prior (AVG Δ from +0.25 to +0.70 points), whereas Fixed $R = 5$ loses 0.61 points (FullKV and shared AVG 44.73

Table 8: **Table 2 without KV reuse.** Same checkpoints as Table 2, evaluated at $R = 5$. Val. and Wiki are native PPL over full sequences, choice tasks are scored without a cache, and generation keeps all twelve visit-specific KV banks. Table 2 instead scores Val. and Wiki only on the tokens after a 4,096-token prompt that continue a document begun inside it (Appendix D.3), so only the task columns compare directly across the two tables. Untied 12 stacks twelve distinct blocks (0.31B, 1.25B and 4.98B non-embedding parameters at S, M and L, three times the looped models) and has twelve KV banks; it is scored the same way and not bolded.

Training	PPL ↓		Downstream (%) ↑								Generation (%) ↑		
	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM8K	DROP	MBPP+
S scale: (100M, 21.5B tokens)													
Fixed $R=5$	6.50	20.62	27.79	32.57	62.19	47.56	25.77	15.20	78.80	41.41	2.05	15.28	0.00
Fixed PLN-5	6.69	21.47	26.04	32.14	62.13	47.94	24.23	15.60	81.20	41.33	2.27	12.39	0.62
Learned, $\lambda_H=0$	6.60	21.14	26.94	32.09	62.19	45.92	24.66	18.00	81.60	41.63	1.90	12.72	0.25
Learned, $\lambda_H=0.01$	6.60	21.12	27.05	32.11	62.84	45.45	26.02	17.40	81.30	41.74	2.12	13.32	0.62
Untied 12	5.80	18.20	31.67	34.78	64.80	50.93	26.02	15.80	82.30	43.76	1.59	12.50	2.75
M scale: (400M, 85.9B tokens)													
Fixed $R=5$	4.51	12.74	40.19	46.04	67.41	57.24	30.55	19.00	90.20	50.09	15.54	21.85	13.12
Fixed PLN-5	4.66	13.31	39.22	44.01	67.36	55.51	29.61	17.20	88.60	48.79	12.89	17.15	9.12
Learned, $\lambda_H=0$	4.55	12.89	39.08	45.30	67.68	57.32	28.75	18.20	88.40	49.25	14.40	19.35	13.62
Learned, $\lambda_H=0.01$	4.57	12.96	38.48	45.30	67.63	56.86	28.58	16.60	87.40	48.69	14.10	17.72	11.75
Untied 12	4.18	11.67	42.11	49.25	70.13	60.14	30.80	19.20	90.50	51.73	15.92	21.14	16.00
L scale: (1.6B, 343.6B tokens)													
Fixed $R=5$	3.51	8.99	54.55	61.68	73.39	69.53	40.27	25.80	94.10	59.90	50.64	36.10	27.12
Fixed PLN-5	3.58	9.23	54.65	60.29	73.56	69.02	37.71	24.40	93.00	58.95	44.12	33.89	24.50
Learned, $\lambda_H=0$	3.53	9.05	55.42	61.85	72.91	70.08	40.27	23.00	93.10	59.52	47.23	35.20	27.38
Learned, $\lambda_H=0.01$	3.54	9.11	55.27	61.71	73.72	71.55	40.02	25.40	93.40	60.15	50.27	35.01	28.88
Untied 12	3.34	8.36	57.73	64.59	74.37	72.31	42.32	25.00	94.50	61.55	49.58	39.18	30.88



(b) KV cache sharing (PPL ↓)						
Training	WikiText		MixVal		LAMBADA	
	Per-visit	Shared	Per-visit	Shared	Per-visit	Shared
M scale: (400M, 85.9B tokens)						
Fixed $R = 5$	12.19	16.71	4.40	5.76	19.82	20.68
PLN-5	12.75	12.60	4.57	4.56	21.89	21.82
$\lambda_H = 0$	12.34	12.34	4.45	4.49	21.36	21.33
$\lambda_H = 0.01$	12.42	12.37	4.46	4.48	22.92	22.93
$\lambda_H = 0.05$	12.48	12.37	4.50	4.49	22.06	22.05
$\lambda_H = 0.1$	12.64	12.53	4.54	4.53	22.27	22.25

Figure 7: **Entropy trades PPL at the training depth for PPL at larger depths; learned priors keep KV sharing.** M scale. (a) WikiText-103 PPL across test-time depths; the bold tick marks the training mean $R = 5$. (b) PPL with per-visit KV (12 banks) and shared KV (4 banks). Panel (b) scores continuations or last words, so its values differ from panel (a).

and 44.12); PLN-5 gives 44.30 and 44.44. Both layouts score identical requests in BF16 without FA3, and each answer option starts from a fresh cache.

Accuracy intervals use 2,000 paired bootstrap draws per task, keeping task weights equal.

Table 10 records final priors and realized means. The CacheEval sharing penalty decreases monotonically with entropy, from +0.96% to −0.22%, and absolute shared MixVal PPL is lowest at $\lambda_H = 0.01$ (4.48, within 0.01 of $\lambda_H = 0$ and 0.05; Figure 7b). Intervals use 2,000 paired sequence-CE bootstrap draws within each source, transformed by $100(\exp(\Delta\text{CE}) - 1)$; they quantify evaluation uncertainty, not training-seed variability or between-coefficient differences.

Table 9: **Terminal sharing does not lower the task accuracy of any learned prior.** M entropy sweep at $R_{\text{pre}} = R_{\text{dec}} = 5$. Per-task columns give Shared minus FullKV accuracy in points; the interval is a paired, task-stratified 95% bootstrap interval.

Prior	Shared – FullKV (points)							AVG (%)			
λ_H	LAMB.	HS	PIQA	ARC-E	ARC-C	WG	OBQA	FullKV	Shared	Δ	95% interval
M scale: (400M, 85.9B tokens)											
0	+0.06	+0.36	+1.64	+0.76	0.00	+1.90	+0.20	43.97	44.67	+0.70	[+0.31, +1.11]
0.01	+0.02	+0.75	−0.33	+0.93	−0.51	+0.47	+0.40	43.68	43.93	+0.25	[−0.14, +0.62]
0.05	+0.02	+0.69	+0.65	+0.84	+0.09	+0.87	0.00	43.77	44.22	+0.45	[+0.10, +0.80]
0.1	+0.04	+0.50	+0.82	+0.71	+0.26	+0.32	+0.60	43.96	44.42	+0.46	[+0.13, +0.80]

Table 10: **Higher entropy spreads the learned prior and lowers the sharing penalty.** Final priors of the M entropy sweep. $\hat{\mathbb{E}}[R]$ is the mean realized draw and $H(p_\phi)$ is in nats; probabilities and penalties are percentages, with paired 95% bootstrap intervals.

Prior	Final depth distribution						Sharing penalty (%)	
λ_H	$\mathbb{E}_{p_\phi}[R]$	$\hat{\mathbb{E}}[R]$	$H(p_\phi)$	$p_\phi(5)$	$p_\phi(R > 10)$	$p_\phi(R > 16)$	Penalty	95% interval
M scale: (400M, 85.9B tokens)								
0	5.007	5.014	1.067	67.40	0.32	0.010	+0.955	[+0.786, +1.136]
0.01	5.008	5.033	1.592	37.79	0.79	0.042	+0.340	[+0.189, +0.503]
0.05	5.009	5.040	2.091	19.32	2.69	0.156	−0.030	[−0.159, +0.101]
0.1	5.016	5.036	2.293	13.98	5.02	0.488	−0.224	[−0.342, −0.108]

Training trajectories and depth distributions. Figure 8 plots the training loss of the S and M checkpoints in Table 2. Figures 4 and 9 use the S/M checkpoints of Table 2: fixed PLN-5 with independent draws and learned priors with $\lambda_H = 0.01$ (LR 0.002 at S, 0.0015 at M). The learned M checkpoint is the $\lambda_H = 0.01$ curve of Figure 7, whose PLN-5 reference is also the Table 2 run.

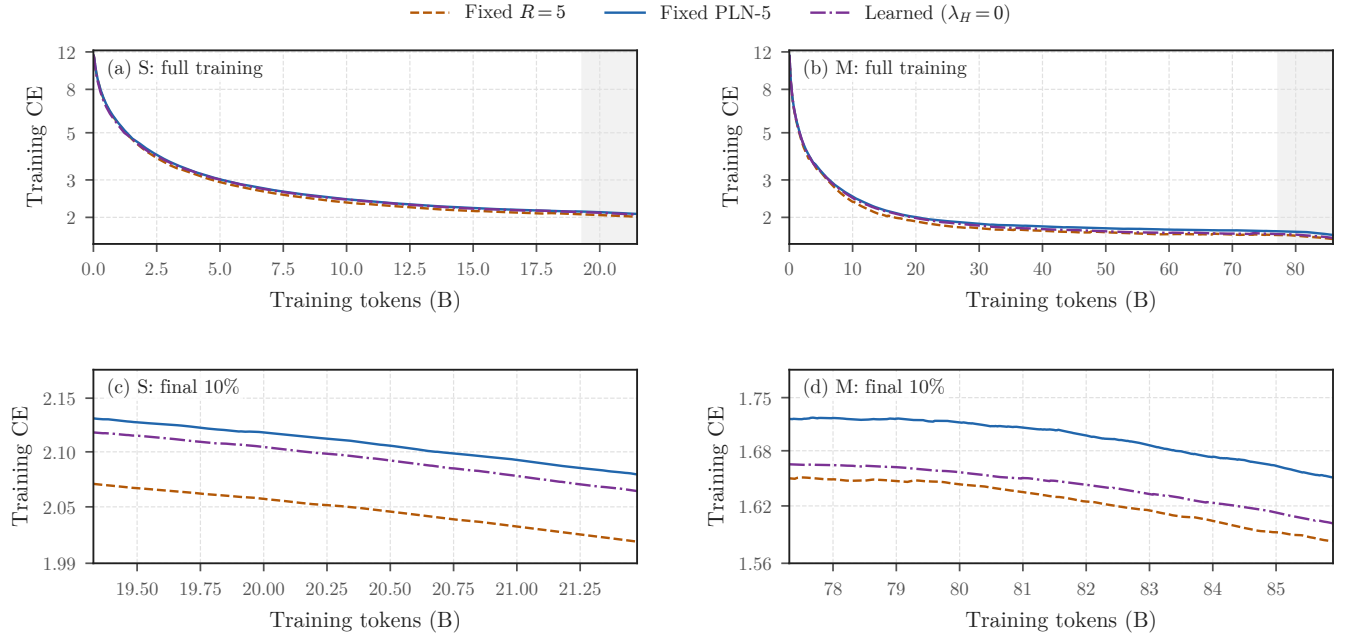


Figure 8: **Training loss for the S/M checkpoints in Table 2.** Fixed PLN-5 draws depths independently. S and M complete 21.5B and 85.9B tokens, respectively; learned priors use $\lambda_H = 0$. Curves show debiased exponential moving averages of optimizer-step CE (decay 0.999), with linear token axes and logarithmic CE axes; the lower panels expand the shaded final 10%. CE is measured at $R = 5$ for fixed-depth training and averaged over sampled depths for the priors. Learning rates and batch layouts follow Appendix B.1.

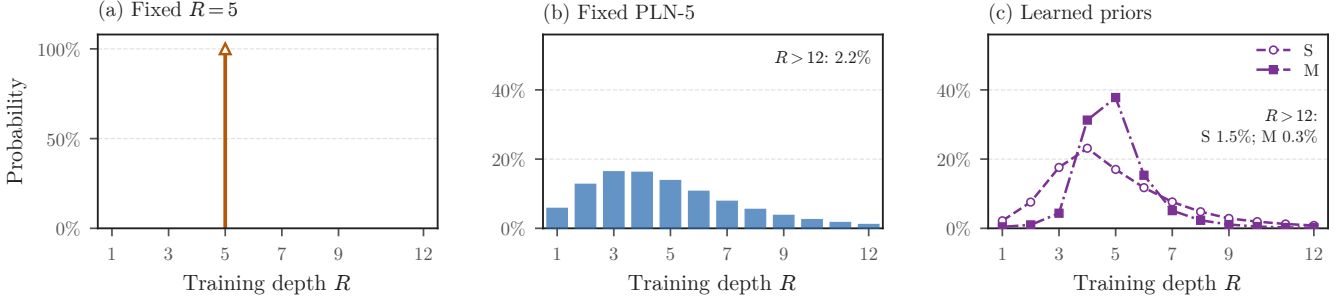


Figure 9: **Depth distributions for the scaling comparison.** Fixed $R = 5$, fixed PLN-5, and learned S/M priors ($\lambda_H = 0.01$) of Figure 4. Depths 1–12 are shown without renormalization; annotations give the remaining mass through depth 64. Panels (b,c) share a probability scale.

Reward and initialization ablations. Table 11 differs from Table 2 in two ways: its S control is an earlier Fixed PLN run with independent draws, and Val. is native PPL on the LR-selection set (Appendix B.1) rather than terminal-KV PPL on MixVAL. Four-task scores use summed answer-token likelihood on full splits (Appendix D.2).

Cached PPL uses four terminal banks and the common CacheEval protocol at $R_{\text{pre}} = R_{\text{dec}} = 5$. At M, the fixed PLN arm uses antithetic draws and the learned priors use independent draws; the arms otherwise share budget, model, data, seed, and optimizer.

Table 11: **Reward and initialization ablations for the learned prior.** Rows name the reward signal. Learned rows use $\lambda_H = 0.01$ and start from capped PLN-5 ($U_{1:k}$: uniform over depths $1, \dots, k$); the M Fixed PLN row uses antithetic draws. FLOPs are in 10^{19} and $\mathbb{E}_{p_\phi}[R]$ is the final prior mean. Val. is PPL on the LR-selection set.

Prior	Training		PPL ↓			Downstream (%) ↑				
	$\mathbb{E}_{p_\phi}[R]$	FLOPs	Val.	Wiki	Cached	HS	ARC-C	ARC-E	PIQA	AVG
S scale: (100M, 21.5B tokens)										
Fixed PLN	5.00	7.31	6.86	21.34	6.52	29.60	22.15	47.06	61.15	39.99
CE	5.00	7.33	6.79	21.17	6.47	29.46	21.80	46.89	61.81	39.99
log(CE)	5.01	7.34	6.85	21.60	6.51	29.69	21.20	46.81	61.75	39.86
exp(CE)	5.01	7.33	6.85	21.59	6.50	29.67	21.03	46.51	61.86	39.77
−exp(−CE)	5.00	7.33	6.80	21.19	6.46	29.50	21.72	47.44	62.35	40.25
CE ($U_{1:10}$)	5.01	7.37	6.87	21.58	6.52	29.67	22.40	47.65	61.92	40.41
CE ($U_{1:64}$)	6.15	11.53	6.94	21.40	6.57	29.42	21.55	46.98	62.24	40.04
M scale: (400M, 85.9B tokens)										
Fixed PLN	5.00	91.01	4.72	13.16	4.49	36.15	25.49	58.86	68.66	47.29
CE	5.01	91.99	4.64	12.85	4.47	36.98	28.24	58.22	69.64	48.27
−exp(−CE)	5.01	91.95	4.63	12.77	4.46	36.76	26.87	58.14	69.37	47.78
CE ($U_{1:10}$)	5.01	92.31	4.64	12.91	4.48	36.89	27.73	59.28	68.72	48.15

With model CE training unchanged, inverse perplexity gives the best cached PPL and task mean among the S learned-PLN arms, while raw CE and inverse perplexity tie on native and WikiText PPL (within 0.02). At M, raw CE leads task accuracy and inverse perplexity leads WikiText and cached PPL, each costing about 1% more FLOPs than fixed PLN. Log and exponential CE have higher S perplexity than either. $U_{1:10}$ leads S task accuracy but has higher PPL; it also changes both initialization and support. $U_{1:64}$ increases compute and worsens validation and cached PPL. These controls match tokens, not compute.

Fixed priors with the learned shape. Table 12 separates the distribution the learned prior reaches from its adaptation during training. At S, the final learned prior moves mass from depths 1–2 to depths 3–5; the capped PLN closest to it in KL has $\sigma = 0.35$. Sampling either this PLN or the frozen final distribution from

the first update, at the learned prior’s LR 0.002, matches or beats the learned prior’s validation PPL (5.67 and 5.63 against 5.67), with a downstream average within 1.1 points. Fixed PLN-5 at the same LR draws the same depths as its Table 2 run and reaches 5.69. At either LR, the learned shape lowers S validation PPL by 0.3–1.1% relative to Fixed PLN-5. The learned prior finds a better fixed shape without a search over distributions; at S, adapting it during training adds no measurable gain. At M, the two rows of Table 2 share one LR; at L, each uses its own selected LR.

Table 12: **At S, fixed priors with the learned shape match the learned prior.** S models evaluated as in Table 2 (terminal KV sharing at $R = 5$). Fixed PLN ($\sigma=0.35$) is the capped PLN closest to the final learned prior in KL; the frozen learned prior samples that final distribution from the first update. The first and last rows are the Table 2 runs; the others match the learned run except for the prior and LR. FLOPs are in units of 10^{19} .

Training			PPL ↓		Downstream (%) ↑								Generation (%) ↑		
Prior	LR	FLOPs	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM	DROP	MBPP+
S scale: (100M, 21.5B tokens)															
Fixed PLN-5 ($\sigma=0.5$)	0.0015	7.292	5.73	20.19	26.41	32.42	61.70	47.56	23.98	16.20	81.00	41.32	1.52	13.29	0.75
Fixed PLN-5 ($\sigma=0.5$)	0.002	7.292	5.69	20.12	27.52	32.40	61.21	46.93	23.98	16.20	79.90	41.16	2.43	14.02	1.00
Fixed PLN ($\sigma=0.35$)	0.0015	7.348	5.71	19.87	26.10	32.29	61.64	45.29	24.83	14.40	82.20	40.96	1.90	10.96	0.38
Fixed PLN ($\sigma=0.35$)	0.002	7.348	5.67	19.75	26.57	32.68	61.70	46.09	24.74	16.60	80.30	41.24	2.20	11.03	0.50
Frozen learned prior	0.0015	7.334	5.71	20.25	26.64	32.84	61.32	46.25	24.83	16.40	82.30	41.51	2.05	11.13	0.38
Frozen learned prior	0.002	7.334	5.63	19.92	27.21	32.54	61.53	44.23	24.74	15.60	78.40	40.61	2.65	13.15	0.12
Learned, $\lambda_H=0.01$	0.002	7.328	5.67	19.91	27.36	32.51	63.00	45.50	25.09	17.20	80.90	41.65	1.59	14.08	0.62

C.2 Input Injection

Models, optimization, batch layouts, and token budgets follow Appendix B.1, and Appendix B.1 gives the learning-rate grids. Equal token budgets need not mean equal compute.

Injection site and KV sharing. DEQ-QKV injects the input into queries, keys, and values. It breaks terminal KV sharing: decoding CacheEval one token at a time (Appendix D.4), its PPL is 16.52, against 6.43–6.69 for the other S schemes and 7.13 on MixVAL without KV reuse (Table 13). At S, generation is near the floor for every scheme (GSM8K at most 2.05%, MBPP+ at most 1.62%; Table 3).

Table 13: **Table 3 without KV reuse.** Same checkpoints as Table 3, evaluated at $R = 5$. Val. and Wiki are native PPL over full sequences, choice tasks are scored without a cache, and generation keeps all twelve visit-specific KV banks.

Model	PPL ↓		Downstream (%) ↑								Generation (%) ↑		
	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM8K	DROP	MBPP+
S scale: (100M, 21.5B tokens)													
DEQ-QKV	7.13	23.83	25.29	30.65	60.99	44.74	24.32	14.20	78.60	39.83	1.36	14.61	0.50
Huginn-Linear	6.89	22.23	25.91	31.40	62.19	43.35	24.06	17.00	79.10	40.43	1.67	11.84	0.50
Parcae-Decay	6.71	21.24	26.02	32.20	61.75	47.26	24.40	16.40	81.40	41.35	2.35	10.94	0.62
OrthoInj	6.60	21.16	27.30	32.38	61.92	45.45	25.34	16.60	82.10	41.59	1.36	13.34	1.00
M scale: (400M, 85.9B tokens)													
Huginn-Linear	4.65	13.33	36.33	43.90	67.85	56.94	29.18	17.40	89.10	48.67	15.09	16.82	8.38
Parcae-Decay	4.62	13.20	37.28	44.37	68.06	57.62	27.82	19.20	88.10	48.92	12.59	19.44	9.00
OrthoInj	4.58	13.06	37.49	44.58	67.57	56.82	29.18	18.20	88.70	48.94	13.57	19.38	10.75
L scale: (1.6B, 343.6B tokens)													
Parcae-Decay	3.58	9.22	53.19	60.32	73.12	68.77	38.82	23.40	93.60	58.75	47.38	33.72	31.25
OrthoInj	3.57	9.21	54.22	60.76	72.58	68.52	37.20	24.60	93.50	58.77	47.01	31.73	27.25

Initialization and evaluation. Native validation CE at S selects the standard OrthoInj initialization of Appendix B.4. M OrthoInj uses the same initialization and one complete 20,480-update checkpoint for every score. Table 3 follows the terminal-KV protocol of Appendix D.3, and Table 13 its no-reuse protocol; generation is excluded from AVG. We keep weight decay at 0.1: in a single-seed S Parcae-Decay control (LR 0.0015, 5,120 updates), zero decay worsens validation and cached PPL from 6.63 and 6.45 to 6.76 and 6.58.

From Parcae-Decay to OrthoInj. OrthoInj differs from Parcae-Decay by two switches: it projects the carryover against the injected vector q , and it drops prelude RMSNorm. Table 14 crosses the two switches at S. Each switch lowers validation PPL on its own: the projection from 6.71 to 6.67, and dropping prelude RMSNorm from 6.71 to 6.63. Their gains add up: the projection lowers PPL by a similar amount with and without prelude RMSNorm (0.04 and 0.03), and OrthoInj, which has both switches, reaches 6.60. OrthoInj also has the highest downstream average; the other three cells lie within 0.03 points of each other.

All four cells train with PLN-5 and independent depth draws, seed 1, common data, optimizer, and batch settings, $\Delta_{\text{init}} = \frac{1}{2} \log 5$, and 5,120 updates; transformer-block RMSNorms are retained throughout. Parcae-Decay and OrthoInj are the S models of Table 3. Each cell uses the LR selected by its own staged screen. The cell without either switch was screened with independent draws over $\{1, 1.5, 2, 3\} \times 10^{-3}$ and selects 2×10^{-3} ; the other three screens used antithetic draws. Setting $\Delta_{\text{init}} = 1$ instead changes selection PPL by at most 0.03 (6.87 vs. 6.84 with prelude RMSNorm and the projection; 6.77 vs. 6.78 for OrthoInj).

Table 14: **Dropping prelude RMSNorm and projecting the carryover each lower perplexity, and their gains add up.** S models trained with PLN-5 and evaluated at $R = 5$ without KV reuse. Norm is prelude RMSNorm and Proj. the carryover projection; LR is each cell’s own selected learning rate; Sel. is PPL on the LR-selection set; AVG is the seven-task mean of Table 13.

Training				PPL ↓			Down. (%) ↑
Recipe	Norm	Proj.	LR	Sel.	Val.	Wiki	AVG
S scale: (100M, 21.5B tokens)							
Parcae-Decay	✓		0.0015	6.86	6.71	21.24	41.35
+ projection		✓	0.0015	6.84	6.67	21.26	41.37
– prelude norm			0.002	6.79	6.63	21.14	41.34
OrthoInj (–norm +proj.)		✓	0.002	6.78	6.60	21.16	41.59

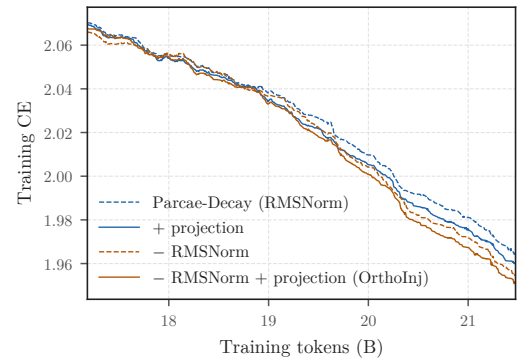


Figure 10: **Both switches lower late-training loss.** Training CE of the four cells of Table 14 over the final 20% of training, as a 512-step rolling median. All cells see the same sampled depths.

Figure 10 shows the late training loss of the four cells. Over the final 256 updates, dropping prelude RMSNorm lowers mean training CE by 0.010 with or without the projection, and the projection lowers it by a further 0.002–0.003. These single-seed comparisons do not estimate seed variance.

Training loss. OrthoInj ends training with the lowest training CE at every scale (Figure 5): over the final 512 updates, its mean is 0.011 (S), 0.005 (M), and 0.003 (L) below Parcae-Decay’s. At S, each run uses the peak learning rate selected by its own screen (Appendix B.1): 2×10^{-3} for OrthoInj and 1.5×10^{-3} for Parcae-Decay. At M, both use 10^{-3} , Parcae-Decay’s selected rate, and OrthoInj’s loss stays below that of both baselines over the whole plotted range. At L, both runs use 5×10^{-4} , and OrthoInj stays below Parcae-Decay over the whole plotted range.

At either M learning rate, OrthoInj has the lower training loss and PPL. OrthoInj’s own M screen selects 1.5×10^{-3} , so we also train both methods at this rate (Table 15). At 1.5×10^{-3} , OrthoInj ends training 0.014 below Parcae-Decay, and Parcae-Decay ends 0.010 above its 10^{-3} run. OrthoInj keeps the lower validation and WikiText PPL at both rates. The downstream average depends on the rate: OrthoInj leads by 0.13 points at 10^{-3} and trails by 0.51 at 1.5×10^{-3} , where it still leads GSM8K and DROP.

Carryover along the injected input. Table 16 measures the component of the carryover Λh_t^r along q_t that motivates OrthoInj (Section 4.2). In Parcae-Decay this component is large and positive: at the fixed point, the carryover has cosine 0.81 (S), 0.87 (M), and 0.91 (L) with q_t , so the input enters the core at $2.2 \times$ (S),

Table 15: At either M learning rate, OrthoInj has the lower training loss and PPL. Train CE is the mean training loss over the final 512 updates; the other columns follow Table 3 (terminal KV sharing, $R = 5$). The Parcae-Decay run at 1.5×10^{-3} trains on 16×8 GPUs with local batch 1 and gradient accumulation 4: its global batch and sampled depths match the other runs, its data order does not.

Injection	Train CE ↓	Val. ↓	Wiki ↓	AVG ↑	GSM ↑	DROP ↑	MBPP+ ↑
Peak learning rate 10^{-3} (Table 3)							
Parcae-Decay	1.602	3.96	12.45	49.03	15.01	20.17	8.25
OrthoInj	1.596	3.94	12.32	49.16	13.95	19.38	7.88
Peak learning rate 1.5×10^{-3} (OrthoInj’s selected rate)							
Parcae-Decay	1.611	3.96	12.41	49.80	14.48	18.63	11.62
OrthoInj	1.598	3.94	12.32	49.29	17.51	21.04	10.25

$2.8 \times$ (M), and $3.4 \times$ (L) its injected size. The factor varies across tokens, from 1.9 to 2.5 at S, from 2.3 to 3.3 at M, and from 2.6 to 4.2 at L between the 5th and 95th percentiles, and no token’s carryover cancels half of the input. Before its projection, the OrthoInj carryover also leans toward q_t (cosine 0.42–0.48); the projection removes this component and fixes the factor at one.

Table 16: In Parcae-Decay, the carryover amplifies the injected input. Carryover Δh_t^r along the injected input q_t at the fixed point of the Table 3 checkpoints: recurrence 32 of a depth-32 forward on 16 MixVAL sequences (131,072 tokens) with attention kept within documents; recurrence 16 agrees to two decimals. Along q is the mean of $\langle \Delta h_t^r, q_t \rangle / \|q_t\|_2^2$, so the input enters the core as $(1 + \text{along } q) q_t$; gain gives its median and 5–95% range over tokens. OrthoInj values precede its projection, which fixes the gain at one.

Injection	$\cos(\Delta h_t^r, q_t)$	Along q	Gain	Gain 5–95%
S scale: (100M, 21.5B tokens)				
Parcae-Decay	0.81	1.19	2.18	1.91–2.49
OrthoInj (before projection)	0.44	0.59	1	1
M scale: (400M, 85.9B tokens)				
Parcae-Decay	0.87	1.76	2.75	2.29–3.26
OrthoInj (before projection)	0.42	0.62	1	1
L scale: (1.6B, 343.6B tokens)				
Parcae-Decay	0.91	2.43	3.43	2.60–4.24
OrthoInj (before projection)	0.48	0.72	1	1

C.3 Truncation, Depth Sampling, and Fixed Points

Cost. Training memory grows linearly with the window: peak allocation is 48, 86 and 124 GB for $b = 5, 10, 15$ at local batch 8. Full BPTT under PLN-5 must also hold the deepest draw; a depth-30 draw at local batch 8 exceeds device memory, and per-layer recomputation brings it to 28 GiB at about $1.3 \times$ the step time of $b = 10$. At a fixed depth $R = 5$, $b = 2$ trains $1.32 \times$ faster than full BPTT. In a Small PLN-25 model at $R = 16$, windows $b = 5$ and $b = 10$ lower saved activations from 3.95 to 1.52 and 2.63 GiB, which stay constant for fixed b once $R \geq b$, while the core gradient at $b = 10$ keeps a median cosine similarity of 0.978 with full BPTT.

With depth sampling, windows of four or more recurrences match full BPTT. Table 17 varies only the backward window under IID PLN-5 draws. At M, every window $b \geq 4$ lies within 1% of full BPTT in WikiText perplexity at $R = 5$, on either side of it, while shorter windows lose 1.6% ($b = 3$), 5.5% ($b = 2$) and 15% ($b = 1$). At S, $b = 5$ and $b = 10$ lie 0.8% and 1.5% above full BPTT. For every window, including $b = 1$, perplexity at $R = 64$ is below that at $R = 5$ and terminal-KV sharing is lossless, so truncation lowers quality without preventing fixed points.

Truncation as a fixed-point shaper. Table 18 fixes the training depth, so depth sampling cannot shape fixed points. Full BPTT then degrades beyond the training depth and loses 21% perplexity under terminal-KV sharing. Backpropagating through only the last two recurrences removes both, with a random or a zero

Table 17: **With depth sampling, windows of four or more recurrences match full BPTT.** Models trained with IID PLN-5 draws. S rows share one LR. Each M window uses the LR selected from $\{0.001, 0.0015, 0.002, 0.003\}$ by a staged sweep (2,048, 4,096 and 20,480 steps); full BPTT is shown at two LR. Terminal KV gives the change in cached PPL from full to terminal KV at $R_{\text{pre}} = R_{\text{dec}} = 5$.

Training		WikiText PPL ↓		Terminal KV
Window b	LR	$R=5$	$R=64$	Δ (%)
S scale: (100M, 21.5B tokens)				
5	0.0015	21.32	21.09	−0.15
10	0.0015	21.47	21.21	−0.16
full (64)	0.0015	21.15	20.94	−0.09
M scale: (400M, 85.9B tokens)				
1	0.001	15.23	15.12	−0.15
2	0.001	13.97	13.85	−0.06
3	0.001	13.46	13.30	−0.29
4	0.001	13.33	13.19	−0.15
5	0.001	13.37	13.18	−0.24
6	0.0015	13.17	13.00	−0.25
7	0.001	13.32	13.16	−0.28
8	0.001	13.32	13.15	−0.31
9	0.001	13.34	13.17	−0.33
10	0.001	13.21	13.05	−0.22
full (64)	0.001	13.25	13.09	−0.21
full (64)	0.0015	13.27	13.10	−0.34

initial state, at a cost of about 0.07–0.10 CE at the training depth. The stop-gradient removes credit for iteration-specific computation in the detached prefix: the window is trained only to map whatever state the model reaches to a good prediction, which favors states that further recurrences leave unchanged. Without input injection, the prefix state is the only carrier of the input, so this pressure erases it: truncation then neither fits the training depth nor yields a fixed point. A two-term endpoint Neumann estimate, which uses only the Jacobian at the endpoint, collapses from initialization at every LR, because no fixed point exists early in training.

Relation to prior uses of truncation. Earlier looped models that extrapolate stably also truncate back-propagation, but in each case together with another source of depth variation. Progressive training detaches a prefix of random length, so the window lands at a random depth, and mixes in a full-depth loss [52]; its motivation, discouraging iteration-specific behavior, matches the mechanism above. The looped transformers of Yang et al. [55] detach all but the last b loops, but grow the loop count by a curriculum and supervise every loop in the window; they find that larger windows help reach a fixed point. Huginn backpropagates through the last eight recurrences of a sampled depth [2]. Both Bansal et al. [52] and Yang et al. [55] also find input injection necessary. Table 18 removes these sources of depth variation: the depth is fixed, the window is the last two recurrences, and full BPTT is the control.

C.4 Recurrent-State and Cache Dynamics

This section gives the protocol behind the tokenwise convergence of Section 2 and measures how far hidden states and KV caches settle at larger depth.

Tokenwise dynamics of trained checkpoints. Figure 2 uses the Small Fixed PLN-5 checkpoint of Table 2 at step 5,120. We take eight frozen samples, one from each of the first eight validation sources in alphabetical order; each sample is the first 2,048 tokens of the source’s first packed sequence, with document boundaries and causal masks preserved. The unsmoothed heatmap in panel (a) shows the first document, which ends at position 1,003; panel (c) pools all eight samples at document-local positions below 1,024.

State and convergence. Each continuous 64-update trajectory uses native normal initialization, executes the prelude and coda once, and generates no tokens. The relative-change diagnostic in Section 2 uses

Table 18: At a fixed training depth, truncation shapes fixed points when the input is injected. S models trained at fixed $R = 5$ with diagonal (Diag.) or no injection, each with its own screened LR. CE is native validation CE at $R = 5$. Without injection, the loop starts from the prelude output, with embeddings and prelude frozen. The two-term endpoint Neumann estimate collapsed at all five LR within 1,024 steps.

Training			Val.	WikiText PPL ↓		Terminal KV
Init.	Injection	Backward	CE ↓	$R=5$	$R=64$	Δ (%)
S scale: (100M, 21.5B tokens)						
Random	Diag.	Full ($b=5$)	1.894	20.62	26.09	+21.9
Random	Diag.	TBPTT $b=2$	1.961	22.29	22.71	+2.9
Zero	Diag.	Full ($b=5$)	1.886	20.76	26.06	+20.7
Zero	Diag.	TBPTT $b=2$	1.984	23.01	23.26	+1.3
Random	Diag.	Neumann, $b=2$		collapsed at every LR		
Prelude	None	Full ($b=5$)	2.345	39.85	9349	+232
Prelude	None	TBPTT $b=2$	4.971	1046	37860	+1890

states after both core blocks and exit normalization, before the coda, with nonzero reference states and FP32 measurements on BF16 states. Under this criterion, Figure 2 sets $R_{\text{obs}} = 32$ and $\tau = 2\%$, requiring a sub-threshold suffix of at least four updates; Table 19 sets $R_{\text{obs}} = 64$. Tokens that do not meet the criterion have no observed convergence depth and leave display gaps. The logarithmic color scale is clipped to $[10^{-4}, 1]$, and the gray initialization row is unclipped.

Positional association. The zoomed panel (b) shows the latest-converging token. Panel (c) includes all qualifying positions, with medians in nonoverlapping 128-position bins: 13,707 positions qualify and two do not. For the correlation in Section 2, ties use average ranks; per-sequence values range from -0.10 to 0.47 . No independent-token significance test is applied.

Table 19: **Hidden states and KV caches settle with depth.** S models trained with PLN-5 and PLN-25. **A:** a token is converged if every later update through recurrence 64 stays below the threshold for at least four updates; onset is the median over converged tokens. **B:** relative pre-RoPE KV change between recurrences $r - 5$ and r for PLN-25, with sequence-bootstrap intervals. Values are percentages except onset, which counts recurrences.

A: Adjacent hidden-state changes and converged-token coverage					
Mean depth	Median $\rho_{64,t}$	Converged at 1% (%)	Converged at 2% (%)	Onset at 2%	
5	0.33	99.74	99.93	7	
25	0.23	97.39	99.09	11	
B: KV changes over five recurrences; PLN-25					
Recurrence r	Reference $r - 5$	Median KV change	95% interval	KV norm / norm at $r = 5$	
25	20	1.23	[1.22, 1.24]	97.48	
30	25	0.85	[0.84, 0.86]	97.48	
160	155	0.59	[0.58, 0.60]	97.48	

Huginn convergence measurements.

Adjacent hidden-state changes. Table 19A applies the initialization, precision and convergence protocol in Appendix C.4 through update 64 over all 16,384 positions; onset medians are computed over qualifying tokens only. The mean-5 and mean-25 models use LR of 1.5×10^{-3} and 2×10^{-3} , respectively, and also differ in training recipe, so the two rows are not a controlled comparison of depth. Within 64 updates, more than 97% of tokens meet the 1% criterion in both models.

Five-recurrence KV changes. Panel B stacks the pre-RoPE keys and values of the banks C^r from both physical core layers into one matrix every fifth recurrence. On each of 1,216 MixVAL sequences (Appendix D.1),

we compute

$$\rho_r^C = \frac{\|C^r - C^{r-5}\|_F}{\sqrt{\frac{1}{2}(\|C^r\|_F^2 + \|C^{r-5}\|_F^2)}}. \quad (25)$$

The table reports sequence medians with pointwise 95% intervals from 10,000 sequence-bootstrap resamples; norm ratios use each sequence’s $\|C^5\|_F$ as the reference. The five-recurrence KV change falls from 1.23% at $r = 25$ to 0.59% at $r = 160$, below the 2% single-update threshold of Section 2 even though it spans five recurrences, while the KV norm stays at 97.48% of its $r = 5$ value, so the remaining change occurs at nearly constant norm. Each continuous 160-recurrence trajectory uses two core-block applications per recurrence without token generation. These measurements characterize state and cache changes; they do not establish global contraction, a causal training-prior effect, or the quality of residual-based stopping.

C.5 Terminal KV Sharing at Deeper Recurrence

Table 20 compares terminal KV sharing with the full per-visit cache on the Table 2 checkpoints at $R = 5, 16$, and 32, where the full cache holds 12, 34, and 66 banks and sharing keeps 4. Sharing raises MixVAL PPL by at most 0.85% at $R = 5$ and by at most 0.11% at $R = 16$ and 32, so its cost shrinks as recurrence deepens, consistent with Lemma 1. With sharing, GSM8K accuracy lies within 3.2 points of the full cache, with no consistent sign. Both caches show the same depth trend: Fixed PLN-5 improves from $R = 5$ to $R = 16$ at every scale, whereas the PPL of the L learned prior rises beyond $R = 5$ while its GSM8K accuracy grows.

Table 20: **Terminal KV sharing stays close to the full cache as recurrence deepens.** Table 2 checkpoints evaluated at depth R with the full per-visit cache ($2 + 2R$ banks) or terminal KV sharing (4 banks), under the protocol of Table 2: Val. is MixVAL PPL on tokens that continue a document begun in the 4,096-token prefill, and GSM8K decodes with the given cache. Δ : relative change in Val. PPL from full to terminal KV.

Prior	Full		Val. PPL ↓			GSM8K (%) ↑	
	R	banks	Full	Terminal	Δ (%)	Full	Terminal
S scale: (100M, 21.5B tokens)							
Fixed PLN-5	5	12	5.733	5.734	+0.01	2.20	1.52
	16	34	5.719	5.719	+0.01	1.52	1.59
	32	66	5.720	5.720	+0.01	1.29	1.36
Learned, $\lambda_H=0.01$	5	12	5.666	5.670	+0.08	1.90	1.59
	16	34	5.661	5.661	+0.01	2.20	2.27
	32	66	5.663	5.664	+0.01	1.97	2.20
M scale: (400M, 85.9B tokens)							
Fixed PLN-5	5	12	3.996	3.997	+0.04	12.89	13.72
	16	34	3.979	3.979	+0.01	14.78	14.78
	32	66	3.981	3.981	+0.00	14.40	14.71
Learned, $\lambda_H=0.01$	5	12	3.917	3.926	+0.23	14.86	15.62
	16	34	3.929	3.930	+0.02	16.68	16.98
	32	66	3.935	3.936	+0.02	16.22	17.13
L scale: (1.6B, 343.6B tokens)							
Fixed PLN-5	5	12	3.092	3.095	+0.10	44.43	47.61
	16	34	3.078	3.078	+0.01	50.42	50.11
	32	66	3.080	3.080	+0.00	50.42	51.18
Learned, $\lambda_H=0.01$	5	12	3.047	3.073	+0.85	50.42	47.92
	16	34	3.137	3.141	+0.11	48.75	50.42
	32	66	3.158	3.159	+0.04	52.24	51.63

At M scale, Table 21 adds the full downstream battery at $R = 5$ and 16: sharing moves the seven-task average by at most 0.34 points at $R = 5$ and 0.04 points at $R = 16$.

C.6 Independent Prefill and Decode Budgets

Terminal KV sharing lets prefill depth R_{pre} and decode depth R_{dec} vary independently: every decoding visit to a physical attention layer reads the same terminal bank, so no decoding visit needs a matching

Table 21: **Terminal KV sharing matches the full cache at $R = 5$ and $R = 16$.** M-scale (400M) checkpoints of Table 2 evaluated at depth R with the full per-visit cache ($2 + 2R$ banks) or terminal KV sharing (4 banks), under the protocol of Table 2. Bold: the better cache within each pair. Table 20 adds $R = 32$ and the S and L scales.

		Cache	PPL ↓		Downstream (%) ↑								Generation (%) ↑		
Prior	R	(banks)	Val.	Wiki	LAMB.	HS	PIQA	ARC-E	ARC-C	OBQA	SciQ	AVG	GSM	DROP	MBPP+
Fixed PLN-5	5	Full (12)	4.00	12.57	39.18	44.05	67.25	55.47	29.69	17.20	88.50	48.76	12.89	17.14	8.75
		Terminal (4)	4.00	12.55	39.69	44.64	67.68	56.61	29.78	16.40	88.60	49.06	13.72	17.75	10.38
	16	Full (34)	3.98	12.37	40.21	44.70	67.63	56.48	29.52	17.60	88.50	49.23	14.78	18.88	7.88
		Terminal (4)	3.98	12.37	40.31	44.63	67.79	56.44	29.35	17.80	88.50	49.26	14.78	19.56	7.88
Learned, $\lambda_H=0.01$	5	Full (12)	3.92	12.24	38.40	45.31	67.74	56.69	28.84	16.80	87.30	48.73	14.86	17.92	12.62
		Terminal (4)	3.93	12.28	38.58	46.17	67.41	57.58	28.75	17.00	88.00	49.07	15.62	17.11	10.38
	16	Full (34)	3.93	12.18	37.88	45.91	67.57	57.15	28.84	18.40	87.90	49.09	16.68	19.41	10.38
		Terminal (4)	3.93	12.18	37.78	45.94	67.46	57.11	28.84	18.20	88.00	49.05	16.98	19.35	8.88

prefill visit. R_{pre} controls only how refined this bank is, while R_{dec} controls how many recurrences each new token receives. Under the convergence conditions of Lemma 1, deeper prefill moves the bank toward its fixed point, so decoding should improve with R_{pre} and then saturate. Figure 11 confirms this for Small checkpoints trained at mean recurrence 5 and 25 once the decode depth is at least 5 (at least 2 for the mean-5 model): beyond $R_{\text{pre}} = 5$, cached log-perplexity changes by at most 0.01 at every decode depth. At the shallowest decode depths, a matched shallow prefill does better; for the mean-25 model at $R_{\text{dec}} = 2$, $R_{\text{pre}} = 1$ gives 2.89 against 3.13 for $R_{\text{pre}} \geq 5$. Decode depth matters more, especially for the mean-25 model, where raising R_{dec} from 1 to 10 lowers log-perplexity from 3.47 to 1.82 before it plateaus.



Figure 11: **Prefill gains saturate by $R_{\text{pre}} = 5$, and decode depth matters more.** Cells give the cached log-perplexity of Small checkpoints trained at mean recurrence 5 (left) and 25 (right), on CacheEval with terminal KV sharing, 1,024 prefix tokens, and continuation positions 2–512.

Both grids use checkpoints with Parcae w/o norm injection, reuse the terminal history, and keep the current token’s KV local to each visit, with $2 + 2R_{\text{pre}}$ block applications per prefill token and $2 + 2R_{\text{dec}}$ per incremental input. Figure 11 evaluates $R_{\text{pre}}, R_{\text{dec}} \in \{1, 2, 5, 10, 15, 20, 25, 32\}$ on CacheEval. KV allocation stays constant across each grid’s 64 settings.

C.7 Distilled Prefill

We first evaluate teacher-initialized students on the full battery of Section 5.5 (Appendix C.7.1) and then distill a deeper $R = 25$ teacher (Appendix C.7.2).

C.7.1 Teacher-Initialized Students under the Full Battery

Evaluation. Table 5 and Tables 22 and 23 use the data, requests, and scorers of Table 2. The prefill path processes the prompt, and the teacher at $R = 5$ processes everything after it while reading the four terminal

KV banks the path produced. For Val. and Wiki, the path prefills the prompt part of the document that spans the first 4,096 tokens of each 8,192-token sequence (1,216 and 38 sequences), and the teacher scores that document’s remaining tokens (Appendix D.3). For the seven choice tasks, the path prefills every context token but the last, and the teacher scores the last context token and the candidate continuation. For GSM8K, DROP, and MBPP+, the path prefills the prompt and the teacher decodes with the settings of Appendix D.5. The teacher rows at $R = 3$ and $R = 1$ prefill with the teacher stopped after that many recurrences and keep each physical layer’s final-recurrence bank, as along the prefill-depth axis of Figure 11. Untied 4 and the standalone student decode themselves with their own KV. The teacher row therefore uses terminal KV throughout and equals the $\lambda_H = 0.01$ rows of Table 2; with the full cache, its downstream averages are 41.74 on Small and 48.69 on Medium (Table 8). Latencies are paired means of six warmed repetitions at batch eight, timed in the teacher’s process; Untied 4 is timed the same way in a separate job.

Teacher and student. The Small and Medium teachers in Section 5.5 are the learned-prior models of Table 2 ($\lambda_H = 0.01$) with Parcae input injection and no prelude RMSNorm. They were trained on 21.47B and 85.90B tokens with learning rates 2×10^{-3} and 1.5×10^{-3} , respectively. The Large teacher is the 1.6B model of the same table, trained on 343.60B tokens at 10^{-3} . Targets are pre-coda states after five recurrent visits. The two-layer students match their teachers’ widths (51.9M parameters on Small, 207.6M on Medium, and 830.5M on Large). Neither student shares weights between its two layers. Equation (4) uses energy normalization calibrated on 64 training rows.

Student training. At each scale, held-out hidden-state MSE on 128 prompts selects among learning rates 3×10^{-4} , 10^{-3} , 3×10^{-3} , and 6×10^{-3} . All run 128 updates, the top two continue to 512, and the selected 10^{-3} reaches 2,560 updates on Small and 10,240 on Medium. The cosine schedule uses eight warmup updates and reaches 10^{-4} at update 2,048, retaining that rate thereafter. All stages use GBS 256 at sequence length 8,192, so each final student sees one quarter of its teacher’s training tokens (5.369B on Small and 21.475B on Medium); training excludes held-out rows.

Teacher initialization. Teacher-initialized students add a $d \times d$ input map to the two-block student (2.36M parameters on Small, 9.44M on Medium, 37.75M on Large). It is set to $\text{diag}(\Delta)W$, so it outputs the injected input q of equation (7), and the two blocks copy the recurrent core. Because the teacher’s initial state is small (standard deviation $d^{-1/2}$ per coordinate), the untrained student approximately reproduces the teacher’s state after one recurrence. They train for the same 2,560 and 10,240 updates at GBS 256 as the randomly initialized students, directly at learning rate 10^{-3} , the rate selected for those students. A control with the same input map, objective, and rate but random initialization (random, +KL) isolates the initialization. The Large student has no such control; it trains for 40,960 updates, one quarter of its teacher’s tokens (85.90B), with the same rate and schedule. The KL loss is the second term of equation (4) and has weight one.

Results. Teacher initialization improves the prefill path (Table 22). With the hidden loss alone, it raises the Medium downstream average from 45.45 to 46.88, GSM8K from 5.53 to 8.42, and MBPP+ from 0.25 to 5.38. On Small, where generation is near zero for every model, it raises DROP from 11.51 to 14.25. With the KL term in both, it raises the downstream average from 45.66 to 46.77 on Medium and from 39.96 to 40.63 on Small, and Medium MBPP+ from 2.50 to 6.12. Adding the KL term, as in the main recipe, keeps the downstream average within 0.1 point and raises all three generation scores on both scales; on Medium, GSM8K rises from 8.42 to 8.95, DROP from 15.21 to 16.20, and MBPP+ from 5.38 to 6.12. Feeding the student’s own KV to the teacher (KV direct) costs 2–7 downstream points relative to +1 rec. Running the student without the teacher (standalone) depends most on the KL loss: it raises the Medium average from 40.64 to 43.75 and lowers Val. PPL from 7.08 to 5.88. Even then, neither use beats Untied 4, which has the

Table 22: **Teacher initialization helps the prefill path, and the KL loss raises its generation scores.** Same battery as Table 5, $R = 5$; AVG is the mean over its seven choice tasks. *+1 rec.* applies one teacher recurrence to the student state (our path; the teacher-init +KL row is the main recipe), *KV direct* feeds the student’s own KV to the teacher, and *Standalone* decodes with the student alone. +KL adds the second term of equation (4).

Prefill path			PPL ↓		Downstream (%) ↑		Generation (%) ↑			Latency ↓
Use	Init	Loss	Val.	Wiki	LAMB.	AVG	GSM	DROP	MBPP+	8K (ms)
S scale: (100M, 21.5B tokens)										
Teacher			5.67	19.91	27.36	41.65	1.59	14.08	0.62	195.0
Untied 4			6.57	25.19	23.95	39.45	1.97	11.39	0.12	100.7
+1 rec.	random	hidden	5.77	20.42	24.24	40.44	1.90	11.51	0.12	125.4
	random	+KL	5.76	20.32	20.88	39.96	1.52	12.44	0.00	125.3
	teacher	hidden	5.72	20.24	24.63	40.73	0.83	14.25	0.38	126.3
	teacher	+KL	5.71	20.16	23.56	40.63	1.74	14.41	0.62	125.9
KV direct	teacher	hidden	6.08	21.36	18.86	38.68	1.29	10.44	0.00	105.1
	teacher	+KL	5.99	21.08	18.96	38.71	1.14	11.28	0.00	104.9
Standalone	teacher	hidden	7.81	30.64	19.41	38.00	2.27	9.41	0.00	100.1
	teacher	+KL	6.97	26.73	21.35	38.22	2.12	9.92	0.00	99.9
M scale: (400M, 85.9B tokens)										
Teacher			3.93	12.28	38.58	49.07	15.62	17.11	10.38	495.8
Untied 4			4.68	15.73	31.19	44.08	2.35	13.12	2.50	229.8
+1 rec.	random	hidden	4.04	12.69	29.15	45.45	5.53	13.20	0.25	295.5
	random	+KL	4.02	12.57	29.52	45.66	7.05	15.36	2.50	295.3
	teacher	hidden	3.98	12.51	33.22	46.88	8.42	15.21	5.38	299.1
	teacher	+KL	3.97	12.44	31.86	46.77	8.95	16.20	6.12	299.3
KV direct	teacher	hidden	4.35	13.66	9.99	39.65	2.12	7.21	0.00	235.6
	teacher	+KL	4.60	14.27	15.85	40.81	1.29	9.39	0.00	235.2
Standalone	teacher	hidden	6.24	22.14	22.67	40.64	1.97	11.22	0.38	226.6
	teacher	+KL	5.14	17.56	29.40	43.75	2.58	12.47	1.00	226.0

same physical depth.

C.7.2 A Deeper Teacher at $R = 25$

Distilling a deeper endpoint should save more prefill time. We trained Small and Medium learned-prior teachers ($\lambda_H = 0.01$) whose PLN prior has mean 25, on the same 21.47B and 85.90B tokens at learning rates 2×10^{-3} and 1.5×10^{-3} , and evaluate them at $R = 25$. Their students follow the $R = 5$ recipe and budget. Table 23 places them next to the $R = 5$ results.

The savings grow with depth: at 8K, distilled prefill is $4.3\times$ faster than the $R = 25$ teacher on Small and $5.4\times$ on Medium, compared with $1.54\times$ and $1.66\times$ at $R = 5$. The quality cost grows faster. The teacher-initialized student trails its $R = 25$ teacher by 2.4 and 3.9 downstream points on Small and Medium, compared with 0.9 and 2.2 at $R = 5$, and it loses most of the deeper Medium teacher’s generation gain (GSM8K 21.38 vs. 4.32). Teacher initialization matters more at this depth: on Medium it raises the downstream average from 42.09 to 45.54. A single recurrence no longer suffices: its downstream average is 33.84 on Small and 32.95 on Medium.

As a result, the $R = 25$ student is both slower and less accurate than the $R = 5$ student at each scale (151.0 vs. 126.3 ms and 39.25 vs. 40.73 on Small; 342.3 vs. 299.1 ms and 45.54 vs. 46.88 on Medium). Its lower Small PPL comes from the stronger $R = 25$ decoder, which scores every continuation token itself; relative to its own teacher, the $R = 25$ student’s PPL gap is $1.7\text{--}3.5\times$ the $R = 5$ gap on both scales. It is faster than the $R = 5$ teacher but falls well below it. Hidden-state error does not explain the gap. The depth-25 endpoint lies farther from the prelude output on both scales (copying the input leaves a relative error of 0.945 vs. 0.856 on Small and 0.997 vs. 0.942 on Medium). The Small students fit it less well (held-out relative MSE 0.070–0.100 vs. 0.058), but the Medium students fit it better (0.011–0.017 vs. 0.036) and still lose more

Table 23: **Distilling a deeper endpoint saves more prefill time but loses more quality.** Mean-25 learned-prior teachers at $R = 25$ next to the $R = 5$ teachers of Table 5, same battery; AVG is the mean over its seven choice tasks. All students use the hidden loss alone at both depths, unlike the main recipe; standalone students decode themselves. The M $R = 25$ teacher-initialized rows reuse the timing of the randomly initialized student.

Prefill		PPL ↓		Downstream (%) ↑		Generation (%) ↑			Latency ↓
Teacher R Path		Val.	Wiki	LAMB.	AVG	GSM	DROP	MBPP+	8K (ms)
S scale: (100M, 21.5B tokens)									
5	Teacher	5.67	19.91	27.36	41.65	1.59	14.08	0.62	195.0
	One recurrence	5.95	21.02	19.29	38.05	1.52	10.86	0.00	110.7
	Distilled, random init	5.77	20.42	24.24	40.44	1.90	11.51	0.12	125.4
	Distilled, teacher init	5.72	20.24	24.63	40.73	0.83	14.25	0.38	126.3
	Standalone	7.81	30.64	19.41	38.00	2.27	9.41	0.00	100.1
25	Teacher	5.41	18.59	28.82	41.66	2.73	13.99	1.00	648.4
	One recurrence	6.36	22.45	10.44	33.84	1.52	3.44	0.00	132.7
	Distilled, random init	5.67	19.46	20.20	38.64	1.67	11.80	0.00	156.1
	Distilled, teacher init	5.57	19.20	22.65	39.25	1.52	11.33	0.12	151.0
	Standalone	10.17	37.43	18.82	36.19	2.73	7.52	0.00	99.2
M scale: (400M, 85.9B tokens)									
5	Teacher	3.93	12.28	38.58	49.07	15.62	17.11	10.38	495.8
	One recurrence	4.23	13.12	13.76	41.19	2.27	8.56	0.12	245.9
	Distilled, random init	4.04	12.69	29.15	45.45	5.53	13.20	0.25	295.5
	Distilled, teacher init	3.98	12.51	33.22	46.88	8.42	15.21	5.38	299.1
	Standalone	6.24	22.14	22.67	40.64	1.97	11.22	0.38	226.6
25	Teacher	3.91	12.17	41.61	49.45	21.38	20.11	11.25	1847.7
	One recurrence	8.95	43.04	0.62	32.95	0.00	3.82	0.00	289.9
	Distilled, random init	4.20	12.92	19.87	42.09	1.74	10.30	0.00	342.3
	Distilled, teacher init	4.05	12.55	30.04	45.54	4.32	14.99	1.38	342.3
	Standalone	8.30	27.84	18.40	39.43	1.82	6.39	0.00	227.1

quality. The deeper teacher’s decoder appears more sensitive to small handoff errors. A stronger teacher need not give a better handoff, consistent with evidence that a larger teacher–student gap can weaken distillation [56, 57]. We therefore report $R = 5$ in the main text.

C.8 Reusing States during Post-Training

This section specifies the rollout-state reuse estimator of Section 3.1, gives the training protocol, and reports the training cost, uncertainty, and diagnostics behind Section 5.4, and a code RL comparison on MBPP+. Saved states are used only before the optimizer step changes the parameters, and missing, stale, or token-mismatched states are rejected.

Forward state and gradient estimator. For a saved endpoint z^R , token sequence x , and input-conditioned recurrent map $F_\theta(z; x)$ of Section 2, Algorithm 1 gives the update. Its only recorded graph is one application of F_θ at the saved endpoint, which includes the prelude through input injection, and $\tilde{z}$ keeps the forward value z^R . With $J_R = \partial_z F_\theta(z^R; x)$ as in Appendix A.2, the hook replaces the readout gradient g with $v = \sum_{j=0}^{b-1} (J_R^\top)^j g$. Neumann-4 ($b = 5$) applies four products, so its sum has five terms, including the identity; Neumann-3 stops after three products (four terms). The number in each Neumann label thus counts vector–Jacobian products. Neumann-4 runs one recurrence forward, four backward passes to the state only, and one full backward pass, all through the same graph. The parameter gradient combines $(\partial_\theta F_\theta)^\top v$ with direct readout derivatives; differentiable attention over saved states preserves cross-token derivatives. Recompute + Neumann-4 reconstructs z^R without gradients before applying this estimator; full BPTT differentiates all six recurrent visits. This endpoint Neumann estimate is approximate and carries no convergence guarantee. Reuse and recomputation differ only in the no-gradient forward pass that recomputation adds. At $R = 6$, the four state products of Neumann-4 are close to the six recurrent visits of full BPTT, so this experiment does not test how the saving scales with depth.

Model, data, and reward. The shared checkpoint in Section 5.4 is the Large learned-prior model of Table 2 ($\lambda_H = 0.01$, 343.6B pretraining tokens). Its input injection is Parcae w/o norm, the injection used by all Table 2 rows. Rollout and evaluation use visit-specific KV. Prompts use the eight-shot chain-of-thought format of Appendix D.5; generation stops at the next question marker, and the binary reward requires the extracted numeric answer to match the reference. Training draws questions from the 7,473-question GSM8K training split in a fixed shuffled order, excluding the eight demonstrations and two probe questions. Evaluation uses a fixed random sample of 500 test questions. We did not verify pretraining decontamination; all arms share the checkpoint, so contamination would shift absolute accuracy but not the comparison between arms.

Policy updates and sampling budget. Each arm samples 16 responses per training question once (temperature 0.8, 512-token cap), forming 200 two-question batches. Dr. GRPO centers each question’s rewards within its group of 16 without standard-deviation scaling and divides the summed token loss by the batch size times the completion cap. Updates use a clipping range of 0.2, no reference KL, the recorded rollout log-probabilities as the importance-ratio denominator, and the rollout temperature during replay. Zero-advantage batches skip the optimizer step, and the others receive one on-policy update, so arms with equal sampling budgets can differ in update and token counts.

Training uses replay microbatch two and gradient accumulation 16, so each update uses 32 responses. AdamW uses LR 10^{-6} , moments (0.9, 0.95), $\epsilon = 10^{-8}$, zero weight decay, gradient-norm clipping at one, and a four-update linear warmup. Evaluation uses the same seed for every endpoint. It draws eight samples per question with the rollout sampler; pass@1 averages the eight outcomes, and pass@8 counts a question as solved if any sample is correct.

Recurrence and task choice. Two measurements favor $R = 6$ for the starting checkpoint. With 64 samples per question at temperature 0.8 on 500 GSM8K training questions, $R = 6$ reaches pass@1/8/64 of 38.1/73.8/91.4%, against 36.2/72.0/89.8% at $R = 5$. With greedy decoding on the test split (Table 24), $R = 6$ also gives the best GSM8K accuracy, while code and reading scores decline slowly through $R = 7$ and fall from $R = 8$. This greedy accuracy (53.68 at $R = 6$) is higher than the sampled pass@1 at temperature 0.8 that Table 4 reports before RL (40.63). Groups of 16 samples contain both a correct and an incorrect response for 78% of the training questions, compared with 65% of the MBPP+ training tasks, so GSM8K gives more learning signal per rollout.

Table 24: $R = 6$ gives the best GSM8K accuracy for the RL starting checkpoint. L learned-prior model ($\lambda_H = 0.01$) with visit-specific KV; GSM8K decodes greedily on the full test split.

R	5	6	7	8	10	15	20
L scale: (1.6B, 343.6B tokens)							
GSM8K (greedy)	50.27	53.68	52.31	52.01	49.96	48.90	49.73
MBPP+ pass@1	28.88	28.75	27.88	25.88	22.50	20.88	20.75
DROP F1	35.01	34.67	33.93	32.39	30.45	30.47	29.15

Quality and uncertainty. Pass@1 gains over initialization in Table 4 are 22.58 ([20.28, 24.90]) points for full BPTT, 19.43 ([17.28, 21.58]) for recomputation, 21.03 ([18.85, 23.23]) for Reuse + Neumann-4, and 20.68 ([18.48, 22.93]) for Reuse + Neumann-3. Reuse + Neumann-4 differs from full BPTT by -1.55 ($[-3.13, 0.05]$) points and from recomputation by $+1.60$ ($[0.08, 3.18]$); Neumann-3 differs from Neumann-4 by -0.35 ($[-2.05, 1.33]$). All 95% intervals use 5,000 paired question-bootstrap draws, resampling questions together with all eight of their responses. They condition on one training seed and so exclude seed variance. Reuse and recomputation compute nearly identical gradients (cosine 0.997–0.999, below), so their $+1.60$ difference indicates the size of run-to-run variation; differences of this size between arms do not show an

Table 25: **Rollout generation dominates RL training time.** Realized counts and times for the GSM8K runs of Table 4. Tokens counts sampled response tokens. Train (min) includes rollout, answer checking, and updates; Update (s) sums scoring, backward, and optimizer time. Prompts whose responses all receive the same reward skip replay, so per-update means fall below the medians in Table 4.

Training	Realized		Time		
Update method	Tokens	Updates	Train (min)	Rollout (min)	Update (s)
L scale: (1.6B, 343.6B tokens)					
Full BPTT	1,009,546	188	65.45	57.42	463.5
Recompute + Neumann-4	880,930	186	61.71	55.50	356.5
Reuse + Neumann-4	886,381	191	57.30	52.97	244.1
Reuse + Neumann-3	858,002	192	58.98	54.95	225.9

estimator effect.

Paired update cost and diagnostics. Timings in Table 4 are per-update medians from the training runs. A paired probe with fixed actions, masks, actor probabilities, and synthetic advantages on two probe questions (two warmups, five repetitions) gives 169.7, 131.6, and 89.2 ms for full BPTT, recomputation, and Reuse + Neumann-4. Whole-model gradient cosines are 0.997–0.999 between reuse and recomputation and 0.76–0.87 between reuse and full BPTT (0.74–0.86 with Neumann-3).

At $R = 6$, the relative state residual averages 0.034 (maximum 0.041 in the Neumann-4 run), larger than near a converged fixed point, so the finite Neumann series truncates more of the full gradient. Reuse’s maximum replay log-probability RMSE is 0.021, and its minimum normalized importance-weight effective sample size is 0.9995. Detached BF16 states stay on the GPU for one batch and use about 2.0 GiB without host transfer; the time to capture them is counted as rollout time. The longest full-BPTT probe peaks at 49.2 GiB after Adam-state allocation.

Total cost and scope. Table 25 reports total costs, which also depend on sampled lengths and nonzero-update counts. Multiple seeds, the full test split, and reuse across updates remain untested.

Transfer to MATH500. We evaluate the starting checkpoint and the four GSM8K endpoints on the 500 MATH500 problems with the Minerva four-shot prompt, a 1,024-token cap, generation stopped at the next problem marker, and Minerva-normalized exact match. Sampling matches the GSM8K evaluation ($R = 6$, temperature 0.8, eight samples per problem), and each problem uses the same seed for every model. RL raises pass@1 by 4.88 ([3.55, 6.22]) points for full BPTT, 3.92 ([2.73, 5.12]) for recomputation, 4.42 ([3.17, 5.67]) for Reuse + Neumann-4, and 5.38 ([3.95, 6.78]) for Reuse + Neumann-3. Reuse + Neumann-4 differs from full BPTT by -0.45 ($[-1.50, 0.68]$) points and from recomputation by $+0.50$ ($[-0.55, 1.50]$), using the paired question bootstrap above.

Code RL on MBPP+. We repeat the comparison on MBPP+ with the same checkpoint, optimizer, and update methods, using $R = 6$, group size 16, and two tasks per update. Prompts use the three-shot MBPP+ format of Appendix D.5, and generation stops at the first test, docstring, or top-level call after the function. The reward is one only if the sanitized solution passes both the EvalPlus base and plus tests, executed in a network-isolated sandbox. The training pool has 273 tasks in a fixed shuffled order, and one pass of 136 two-task batches uses the first 272. These tasks are disjoint from the 100 held-out evaluation tasks and the three demonstrations, with exact prompt and solution duplicates removed.

All methods raise pass@1 from 30.9% to 37–41% (Table 26). Pass@8 moves by at most six points, and each held-out task is worth one point of it; Reuse + Neumann-4 ends five points below full BPTT (64.0 vs. 69.0). Reuse + Neumann-4 differs from full BPTT by -1.50 ($[-4.38, 1.38]$) points of pass@1 and from recomputation by $+2.25$ ($[-1.25, 5.75]$); Neumann-3 differs from Neumann-4 by -1.38 ($[-5.12, 2.25]$).

Table 26: **Reusing rollout states also cuts the update time of code RL.** Same checkpoint, $R = 6$, optimizer, and update methods as Table 4. Training covers 272 MBPP+ tasks once; evaluation draws eight samples per task on 100 held-out tasks. Update is the median scoring plus backward time per update; Train includes rollout, code verification, and updates. Before RL resamples the starting checkpoint with the RL sampler; its pass@1 differs from Table 24 (28.75, independent samples) by 2.1 points (95% interval $[-1.4, 5.8]$).

Training	MBPP+ (%) $\uparrow$		Time $\downarrow$		
Update method	Pass@1	Pass@8	Update (s)	Train (min)	Rollout (min)
L scale: (1.6B, 343.6B tokens)					
Before RL	30.88	63.00	—	—	—
Full BPTT	40.88	69.00	1.03	20.0	14.4
Recompute + Neumann-4	37.12	69.00	0.65	18.9	14.9
Reuse + Neumann-4	39.38	64.00	0.51	21.8	17.8
Reuse + Neumann-3	38.00	68.00	0.48	21.2	17.0

Intervals use the same paired question bootstrap as on GSM8K. Reuse speeds up scoring and backward computation by $1.27\times$ over recomputation and $2.02\times$ over full BPTT. Rollout generation takes 72–82% of training time and code verification another 12–15%, so sampling dominates end-to-end cost, as on GSM8K. The reuse arms are not faster end to end: their rollouts take 17.0–17.8 minutes against 14.4 for full BPTT, which outweighs their 1.5–1.6-minute saving outside rollout.

D Datasets and Evaluation Protocols

We define the shared datasets, target populations, and scoring rules. Appendix C gives each experiment’s checkpoint, execution budget, and any departures from these common protocols. Each cached, split, or reused-state path for evaluation or gradients was checked against a reference (uncached scoring, full autograd, or an analytic oracle) on a subset of inputs before use; the experiment descriptions omit these checks. Unless stated otherwise, results use one training seed, intervals quantify evaluation-sample uncertainty, and “batch eight” means a batch size of eight.

D.1 Mixture-Matched Validation

MixVAL is a fixed mixture-matched split of 64 packed sequences of length 8,192 from each of 19 TxT360 validation sources, totaling 9,961,472 next-token targets. We report $\text{PPL} = \exp(\text{CE})$ from token-weighted teacher-forced cross-entropy, evaluated in BF16 with multi-segment semantics and no generation cache. Document-level separation from the training corpus remains unverified.

D.2 WikiText-103 and Zero-Shot Tasks

Table 27 lists the evaluation tasks.

We concatenate the 2,891 nonempty records of WikiText-103 [30] with newlines. Native curves in Figures 4 and 7 score 310,315 next-token targets in 38 windows of 8,192 tokens at every depth 1–64, using batch two, BF16, FlashAttention-3 (FA3), and state seed 42. PPL exponentiates token-averaged cross-entropy; document-level pretraining decontamination was not performed.

Paired WikiText PPL in Figure 7 uses the same windows and example IDs but scores 271,403 continuation targets after a 1,024-token prefill, excluding the prefill prediction and padding. Both layouts start from an identical per-visit prefill at $R_{\text{pre}} = R_{\text{dec}} = 5$, with batch eight and FA3 disabled; terminal sharing retains each physical layer’s last bank, reducing twelve banks to four.

The main tables share six zero-shot tasks from the Mamba-3 suite [58], using complete labeled splits (LAMBADA test, HellaSwag and PIQA validation, and ARC and OpenBookQA test): LAMBADA OpenAI

Table 27: **Evaluation tasks.** Choice tasks are zero-shot and score candidates by log likelihood (length-normalized for HellaSwag and ARC-C, summed otherwise). Generation tasks decode greedily except MBPP+, which averages eight samples at temperature 0.8.

Task	Split	Metric	Examples	Shots	Used in
LAMBADA (OpenAI)	test	accuracy, last-word PPL	5,153	0	main tables
HellaSwag	validation	accuracy	10,042	0	main tables
PIQA	validation	accuracy	1,838	0	main tables
ARC-Easy	test	accuracy	2,376	0	main tables
ARC-Challenge	test	accuracy	1,172	0	main tables
OpenBookQA	test	accuracy	500	0	main tables
SciQ	test	accuracy	1,000	0	main tables
WinoGrande	validation	accuracy	1,267	0	Appendix C.1, Appendix C.2
DROP	frozen subset	F1	500	3	main tables
GSM8K	test	accuracy	1,319	8	main tables, Appendix C.8
MBPP+ v0.2.0	frozen subset	pass@1	100	3	main tables, Appendix C.8

(5,153), HellaSwag (10,042), PIQA (1,838), ARC-Easy (2,376), ARC-Challenge (1,172), and OpenBookQA (500). HellaSwag and ARC-Challenge normalize continuation log likelihood by character length; PIQA, ARC-Easy, and OpenBookQA sum it. LAMBADA accuracy requires every final-word subtoken to match its greedy prediction; last-word PPL exponentiates the negative mean final-word log probability given the context, summing over subtokens.

Tables 2 and 3 and their no-reuse counterparts add all 1,000 SciQ test questions at $R = 5$, with the support paragraph followed by `Question: {question}` and then `Answer: .` Four answers are scored by summed log likelihood. Table 9 and the entropy-sweep averages in Appendix C.1 instead add the WinoGrande [59] validation split (1,267 examples), scoring the common suffix after each candidate-filled prefix by summed log likelihood. Each seven-task AVG equally weights its accuracies, excluding PPL and generation.

The main distilled-prefill results (Table 5) use this battery, with each prompt split between the prefill path and the teacher decoder (Appendix C.7.1).

Request formatting and pair tokenization use `lm-evaluation-harness v0.4.2` [60], commit 4600d6bf, pinned for this implementation; Mamba-3 does not specify a harness revision. Native scoring uses BF16 with FA3, candidate batch eight, and no BOS, EOS, chat template, or generation cache. State seed 42 and fixed example IDs are shared across candidates and checkpoints. Requests are right-padded into token-determined power-of-two buckets of minimum width 128; batches and bucket shapes stay fixed across models. We keep all ARC questions, including those with three or five choices, and reject rather than truncate inputs over 8,192 tokens.

The reward ablations (Table 11) instead use raw-likelihood scoring and a four-task mean.

D.3 Terminal-KV Evaluation

Tables 2 and 3 and the Teacher row of Table 5 score every checkpoint at $R = 5$ with terminal KV sharing, using the battery’s requests and scorers. At $R = 5$, a token visits the four physical attention layers twelve times: the prelude and coda once each, and each of the two core blocks five times. The cache keeps one bank per physical layer. Because the prelude and coda run once per token, their banks are exact; each core bank keeps only the final-recurrence KV of earlier tokens. A forward pass over a chunk of new tokens runs all five recurrences; at every core visit, the chunk attends to that layer’s bank and to its own KV from the same visit, and after the final recurrence its KV is appended to the bank. From an empty cache this pass

equals the native forward pass and differs only in what it stores; every checkpoint passes this check on MIXVAL prefixes.

Prefill passes every prompt token but the last as one chunk, then the last prompt token alone. For the seven choice tasks, the context is the prompt, and each candidate token is then passed alone, so every scored token reads earlier tokens only through terminal banks. Generation prefills the prompt the same way and decodes one token at a time with the settings of Appendix D.5. For Val. and Wiki, the prompt is the first 4,096 tokens of each 8,192-token sequence (1,216 MIXVAL and 38 WikiText sequences), and attention stays within documents, as in training. PPL covers the tokens after the prompt that continue a document begun inside it, and the prediction from the last prompt token: 42% of the remaining MIXVAL tokens, and every WikiText token, since WikiText sequences contain no document boundary. The prompt part of each such document is prefilled, and its later tokens read it through the terminal banks while attending to one another as in training, so Val. and Wiki test sharing of the prefix only. Tables 8 and 13 evaluate the same checkpoints without KV reuse: Val. and Wiki are native PPL over full sequences (Appendix D.1), choice tasks are scored without a cache, and generation uses $R = 5$ prefill and decoding with twelve visit-specific KV banks and a BOS token.

D.4 Cached-Continuation Perplexity

Cached PPL is teacher-forced next-token perplexity from the native incremental decoder. Prefill’s final-position logits score the first continuation target; appending each reference token once scores the next target. For the scored positions $\mathcal{T}_i$ of sequence i ,

$$\text{CE}_{\text{cache}} = -\frac{1}{\sum_i |\mathcal{T}_i|} \sum_i \sum_{t \in \mathcal{T}_i} \log p_{\theta}^{\text{inc}}(x_{i,t} \mid x_{i,<t}), \quad \text{PPL}_{\text{cache}} = \exp(\text{CE}_{\text{cache}}). \quad (26)$$

Predicted EOS tokens do not stop scoring.

CACHEVAL contains 304 frozen single-document MIXVAL spans (16 per source), each with 1,024 prompt tokens and 512 continuation targets. The cache grows from 1,024 to 1,535 tokens over 511 teacher-forced inputs. Prior, input-injection, and training-depth comparisons score targets 2–512 (155,344 total), using five recurrences for both prefill and decoding; Appendix C.6 varies these budgets independently. Each experiment specifies its cache layout.

D.5 Generation Benchmarks

All generation scores are percentages. DROP [38] uses official F1 on 500 frozen examples (three-shot, greedy), GSM8K [37] uses numeric-answer accuracy on all 1,319 test examples (eight-shot, greedy), and MBPP+ uses estimated pass@1 on 100 frozen v0.2.0 tasks (three-shot, eight samples per task, temperature 0.80). MBPP+ augments MBPP [39] with EvalPlus tests [40]. Generation caps are 64 tokens for DROP and 512 for GSM8K and MBPP+. RL uses the same eight GSM8K demonstrations; Appendix C.8 gives its training split and rollout protocol. Each experiment specifies its model and recurrence budgets.

E Extended Related Work

We position the learned depth prior, OrthoInj, terminal KV sharing, rollout-state reuse, and distilled prefill against related work that Section 6 covers only briefly.

E.1 Learned Depth Distributions and Exits

Ouro, RL-Halting, and our prior all optimize an entropy-regularized expected objective over depths [5, 51]. Ouro takes the exact expectation with per-token exit gates and reuses the gates for early exit.

Closest related work. The closest related work is RL-Halting [51], which also supervises one sampled depth per example and learns the depth distribution with REINFORCE. Its stopping head reads the detached, average-pooled hidden state after each iteration and outputs a hazard π_r , the probability of stopping at recurrence r given no earlier stop, so $p(R = r \mid x) = \pi_r \prod_{r' < r} (1 - \pi_{r'})$. The reward is the negative task loss with a moving-average baseline and entropy coefficient 0.01, and inference stops at the most probable depth. Because the hazards depend on the states along the loop, training unrolls every example to its maximum depth of 20 to form the distribution; the sampled depth selects the supervision but not the compute. A hazard that ignores the state is constant and gives a geometric distribution, so the head cannot start from a prescribed prior such as capped PLN. RL-Halting has no budget term and targets length extrapolation on synthetic tasks; it does not evaluate fixed points or KV sharing.

Our prior is unconditional and used only in training, giving it three advantages over Ouro and RL-Halting. First, it saves compute: it samples one depth per microbatch before the loop and runs only that depth, and the budget term in equation (6) fixes the expected forward compute at $\bar{R}$. Ouro instead runs every step to maximum depth for the exact expectation, and RL-Halting unrolls to that maximum depth to form its state-dependent distribution. Second, it starts from a strong prior: its logits start at the capped PLN prior, so each learned run begins at the fixed-prior baseline, whereas a state-dependent hazard has no setting that reproduces a prescribed prior. Third, it deliberately promotes KV sharing: because the prior does not decide where inference stops, its entropy term extends supervision above $\bar{R}$, where settled prefix context supports terminal KV sharing (Appendix C.1). Table 11 compares its inverse-perplexity reward to RL-Halting’s raw CE.

Learned exits have two limits for fixed-point training. A single exit distribution both stops inference and weights the per-depth training losses; Popescu et al. [61] trace gate failures mainly to the trajectory this joint training induces, and find that fixed-prior depth supervision yields useful trajectories. An exit also stops once prediction suffices, so training need not reach depths beyond prefix convergence, where supervision prepares the model for terminal KV sharing (Section 3.2). Depth uncertainty also learns a global depth distribution, but within a Bayesian objective that marginalizes over depths at inference [48].

E.2 Input Injection and Orthogonal Updates

DEQ-Transformer injects embeddings into Q/K/V; DEQ-TrellisNet injects them before convolutional LSTM gates [14, Section 4]. TrellisNet combines causal convolution, shared depth, and persistent injection [62]. Huginn and Parcae instead inject at the recurrent-core entrance [2, 15].

Orthogonal residual updates [63] also apply a projection inside a residual block, but to the other operand. They keep only the component of a module’s output that is orthogonal to the residual stream, so an update adds new directions without rescaling the stream. OrthoInj (equation (8)) instead projects the carried-over state off the injected input q_t and then adds q_t , so the component along q_t is fixed at every recurrence while the orthogonal part evolves freely. The two constraints act on different quantities: what a block adds, and what the recurrence retains.

E.3 KV Sharing Across Recurrent Depth

Huginn reports zero-shot cyclic reuse of a fixed historical-KV budget, with later recurrent visits overwriting earlier entries [2, Section 6.2]. Ouro’s last-loop reuse during decoding largely preserves GSM8K accuracy with a modest MATH-500 [64] decrease, while prefill sharing degrades performance [5, Section 5.4.2, Table 14]. Even with visit-specific prompt banks, decode-time sharing represents the generated tokens by shared banks.

MELT instead finds zero scores on its Table 5 reasoning benchmarks for all four combinations of first-/last-loop reuse and preserved/shared prompt caches on Ouro-1.4B-Thinking [65, Appendix B]. Its LightEval setup permits 32K-token completions. Differences from Ouro’s few-shot CoT evaluation, alongside incomplete reproduction of the original results, leave the discrepancy unresolved [65, Section 4.1 and Appendix F.1]. MELT adapts the model to a shared cache with gated updates, chunk-wise training, a gradual transition, and attention-aligned distillation; it does not directly evaluate Huginn.

Under a common GSM8K setup without retraining, Schwethelm et al. [9, Appendix C.1, Table 1] find that one shared KV slot per physical layer leaves Huginn accuracy nearly unchanged at 32 recurrences, while Ouro accuracy collapses at four. A separate first-visit slot recovers most of Ouro’s accuracy. These results suggest that cache compatibility depends on the model; Huginn under MELT’s long-reasoning protocol remains unmeasured. These studies test sharing zero-shot or after adaptation. We instead change only the training depth policy, which decides whether terminal sharing is nearly lossless (Table 2); like them, we do not measure long-form reasoning.

E.4 Fixed-Point Reuse

Implicit models reuse a fixed point to warm-start the solver on a related input. DEQ-Flow and StreamDEQ start each video frame from the previous frame’s fixed point [66, 67], and fixed point diffusion models reuse solutions between denoising timesteps [68]. In each case, the reused state initializes a new forward solve. Rollout-state reuse (Section 3.1) instead carries states between the two passes that RL makes over the same tokens, rollout and the policy update. The update scores the sampled tokens from the endpoints saved during rollout, so it does not replay the recurrence, and backpropagates through a single recurrence at each endpoint with a truncated Neumann series (Appendix C.8).

E.5 Distillation for Looped Models

Distillation has trained looped students from non-looped teachers [69] and adapted looped models to cheaper KV caches: MELT adapts Ouro to one gated cache per layer with attention-aligned distillation [65], and Looped Latent Attention distills a codec that compresses KV across loops [70]. LoopFormer trains shorter loop trajectories of the same model to match the final states of the full trajectory [71]. In each case, the student still iterates the recurrent block. Distilled prefill instead trains a separate, non-recurrent student to predict the endpoint of a frozen looped teacher and uses it only for the prompt; decoding keeps the teacher’s recurrence. The closest idea is the learned initializer of HyperDEQ, a small network that predicts the starting point of a frozen DEQ’s fixed-point solver [72]. Our student instead replaces the solver: one teacher recurrence from its prediction builds every KV bank that decoding reads under terminal sharing.